UNIVERSIDADE LaSalle
Viver é evoluir.

NOVA GRADUAÇÃO UNIVERSIDADE LA SALLE

UNIDADE 4

Requisitos de SOFTWARE



INOVAÇÃO E TECNOLOGIA

unidade
4

Gerenciamento de Requisitos de Software

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Identificar como gerenciar requisitos de software.

Editora
LaSalle



INOVAÇÃO
E TECNOLOGIA

unidade
4

Parte 1

Rastreabilidade Bidirecional de Requisitos de Software

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Rastreabilidade bidirecional de requisitos de *software*

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Reconhecer as causas e as consequências das mudanças em requisitos de projetos de *software*.
- Identificar os elementos que compõem uma matriz de rastreabilidade de requisitos de *software*.
- Correlacionar os elementos da matriz bidirecional de requisitos de *software*.

Introdução

No momento em que iniciamos um projeto de *software*, uma coisa é certa: os requisitos vão mudar! Mesmo que todos os cuidados tenham sido tomados no momento da elicitação, análise, especificação e validação dos requisitos, é certo que eles sofrerão mudanças. Mesmo que o analista de requisitos tenha utilizado as melhores técnicas da engenharia de requisitos, e que tenha realizado todas as atividades com maestria, eles mudarão!

Mudarão a qualquer momento e por motivos diversos. O importante é que estejamos preparados para as mudanças e que saibamos como lidar com elas ao longo do ciclo de vida do produto de *software*, de modo que possamos minimizar os custos dessas mudanças e seus possíveis efeitos negativos sobre a qualidade do produto.

Para que possamos analisar o impacto decorrente de uma solicitação de mudança e, posteriormente, planejar a modificação, é necessário que sejam disponibilizadas informações para apoiar essa tomada de decisão. Uma das formas de realizar essa análise é por meio dos registros de rastreabilidade. Identificar e manter a rastreabilidade dos requisitos é uma das atividades da gerência de requisitos.

Neste capítulo você vai ler sobre as causas e consequências das mudanças em requisitos de *software*. Vai também estudar como identificar os elementos que compõem a matriz de rastreabilidade e como correlacioná-los, de modo que possam servir de apoio para a análise do impacto de solicitação de mudanças e outras análises.

1 Causas e consequências das mudanças em requisitos de *software*

Há algum tempo, quando a tecnologia dominante era a do *mainframe* e tudo o que dispúnhamos era de uma tela preta com letras verdes, a vida da equipe de desenvolvimento era mais simples, pois o usuário não tinha familiaridade com a tecnologia e o único contato que ele tinha com computadores era por meio de terminais, nos quais sistemas corporativos eram operados. Não havia interface gráfica, nem celulares, muito menos *smartphones*. A qualidade da experiência do usuário era bastante limitada.

Do nosso lado, linguagens simples como o Cobol, associadas a um banco de dados, davam conta do recado de sistemas comerciais. Não havia a experiência dos computadores pessoais, portanto a variedade de tecnologias era bem menor. Geralmente, as mudanças nos requisitos eram provenientes de mudanças nas regras de negócio. Essa era a realidade até o final da década de 1980.

Nos dias de hoje, no entanto, o desenvolvimento de *software* se tornou uma atividade muito mais complexa. A maioria das pessoas é exposta a uma dezena de aplicativos diariamente, seja na sua vida pessoal, seja na sua vida profissional. A interface gráfica trouxe a facilidade de operação e os celulares trouxeram a mobilidade e noção de instantaneidade. Isso fez com os usuários se tornassem muito mais exigentes em relação à sua experiência com produtos de *software*. Os sistemas, que agora envolvem diversas tecnologias de *hardware* e *software* de forma integrada, tornaram-se mais complexos e difíceis de desenvolver. Os usuários mudam de ideia em um piscar de olhos, pois esse contato com a diversidade de produtos os torna muito mais críticos e, por conseguinte, mais propensos a desejar novos requisitos.

Os desafios para o desenvolvimento de *software* aumentaram não apenas pelo avanço e popularização da tecnologia, mas também porque as mudanças são exigidas em uma velocidade cada vez maior — e esse é um risco para a qualidade de *software*.

Causas das mudanças nos requisitos

As causas para as mudanças nos requisitos vão muito além das mudanças de tecnologia. Devemos ter sempre em mente que o usuário tem um problema que precisa ser resolvido por meio do *software*. O *software* não é um fim em si mesmo, mas é o meio pelo qual o problema é resolvido, seja um problema do mundo dos negócios, seja uma questão ligada ao lazer ou à sua vida pessoal. O usuário entende do problema em si, e não do mundo de desenvolvimento de *software*. E nós, da computação, entendemos do mundo dos *bits* e *bytes*, e não do negócio.

A diferença entre esses dois universos faz com que as atividades relacionas à engenharia de requisitos funcionem como uma espécie de ponte que ajuda a reduzir essa distância. Com a adoção das metodologias ágeis, o processo de comunicação e aproximação entre os dois mundos passou a ser mais valorizado ainda. É mais importante uma comunicação aberta e transparente do que a rigidez de um contrato. A mudança é um princípio básico, não apenas aceito, mas também desejado por quem desenvolve utilizando métodos ágeis. Quando as atividades de elicitação de requisitos são realizadas de forma adequada, as chances de mudança são menores, mas ainda assim podem ocorrer.

Mas afinal, por que os requisitos mudam? Essa é uma pergunta meio estranha nos dias de hoje e talvez ela devesse ser invertida: o que não muda? Em um mundo que está em constante transformação, especialmente no setor de tecnologia, mudar é a regra e não a exceção.

Os requisitos mudam por diversos motivos, além da própria evolução tecnológica:

- os *stakeholders* que participaram da elicitação não tinham todo o conhecimento necessário e os requisitos se mostraram inadequados ou incompletos quando colocados em operação;
- o processo de comunicação falhou, os requisitos foram mal compreendidos durante a elicitação e, no momento da validação, foram rejeitados pelos *stakeholders;*
- o usuário tomou contato com outros *softwares,* o que ampliou sua visão e o levou a solicitar novas funcionalidades ou alteração em funcionalidades existentes;
- o concorrente lançou um novo produto no mercado;
- um fornecedor de tecnologias de apoio (banco de dados, ambientes de desenvolvimento) descontinuou uma versão do seu produto, demandando alterações no produto;

- novas regulamentações foram liberadas e os requisitos de ordem legal precisaram ser alterados;
- e mais um sem-fim de motivos...

Independentemente da causa, os requisitos mudam e temos que abraçar a mudança. O nosso papel, como provedores de tecnologia, é analisar o impacto da mudança, avaliar os riscos e priorizar sua implementação. Isso faz parte do gerenciamento de requisitos, que é uma das atividades da engenharia de requisitos, conforme ilustra a Figura 1.

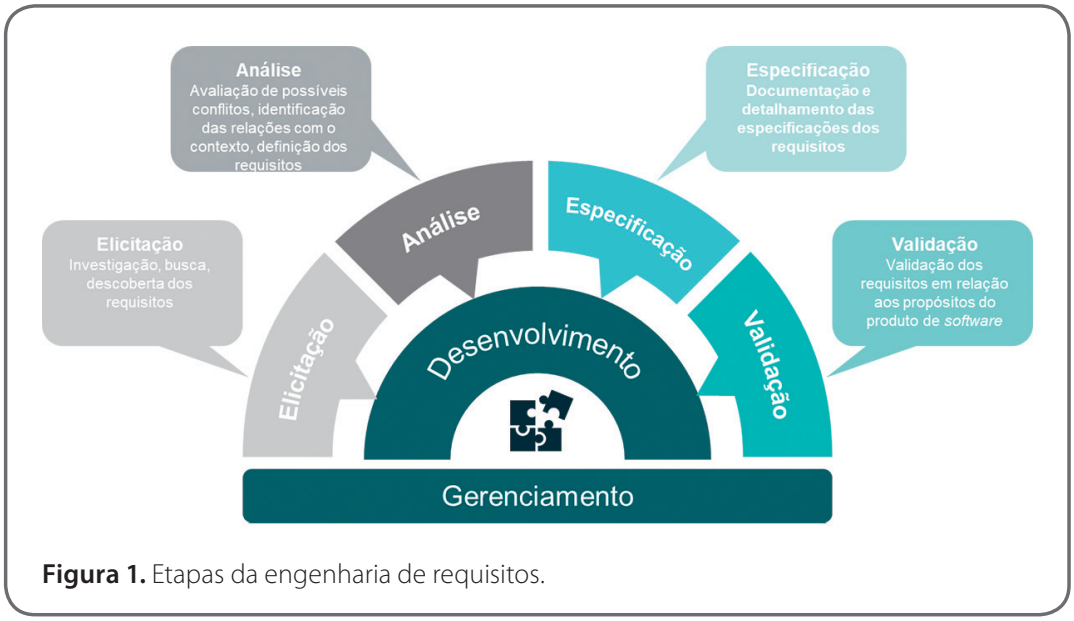


Figura 1. Etapas da engenharia de requisitos.

Como você pode observar na Figura 1, as atividades de elicitação, análise, especificação e validação constituem o núcleo das atividades da engenharia de requisitos. O gerenciamento permeia todas as atividades, tanto da engenharia de requisitos quanto do próprio desenvolvimento em si.

Nosso foco neste capítulo não é sobre o conjunto completo das atividades que estão envolvidas no gerenciamento de requisitos. Vamos focar apenas naquelas relacionadas à rastreabilidade de requisitos.

Consequências das mudanças nos requisitos

Quando um requisito muda, as consequências são as mais diversas e dependem da natureza da mudança em si e do *status* do requisito que é alvo da mudança. Quando estamos realizando as atividades de elicitação de requisitos, é co-

mum que os requisitos ainda estejam voláteis e que sejam refinados até que seja possível considerá-los estáveis e aptos a passar para as próximas etapas. Até aqui, o custo de uma mudança em geral é baixo, pois pouco esforço foi empreendido. Geralmente foram apenas as horas do analista de requisitos e dos *stakeholders* envolvidos na elicitação.

No entanto, à medida que o desenvolvimento avança, os custos para a mudança vão aumentando, pois mais esforço já foi empregado para detalhar o requisito, projetar a sua implementação, implementá-lo, testá-lo e entregá-lo. Quanto mais distante do momento em que o requisito foi capturado a mudança ocorrer, mais custosa ela será. Nesse caso, esforços de mais pessoas da equipe já terão sido dispendidos. Implementar a mudança vai significar refazer todo ou parte do trabalho realizado até o momento.

E que consequências essas mudanças podem trazer? Depende. Mudanças em requisitos de interface, por exemplo, tendem a ser mais fáceis de realizar e serem menos custosas. Já as mudanças que envolvem a arquitetura da aplicação são mais complexas e, consequentemente, mais custosas e arriscadas.

Há requisitos que, quando alterados, impactam poucos elementos e cuja alteração é mais simples. Há outros, no entanto, que impactam uma grande quantidade de outros requisitos e elementos. Em todos os casos, esforços de análise, mesmo que para rejeitar uma mudança, terão que ser dispendidos.



Saiba mais

Um dos casos emblemáticos relacionados a efeitos desastrosos de mudanças mal dimensionadas é o do Boeing 737-300 Max. Ao tentarem reutilizar parte do *software* de um modelo anterior, os projetistas não previram determinadas situações, que foram detectadas apenas na prática e que acabaram ocasionando a morte de 346 pessoas em menos de cinco meses. Como consequência, a empresa precisou paralisar a operação de todas as 387 unidades que estavam em funcionamento em 59 companhias aéreas, gerando um prejuízo de milhões de dólares para a organização (BOEING…, 2019).

Se quiser conhecer mais sobre esse caso, pesquise o vídeo da entrevista da Aero Por Trás da Avaliação com o Lito da Aviões e Músicas. Coloque como palavra-chave "Boeing 737-300 Max". O entrevistado explica em detalhes o que levou a esses dois acidentes aéreos.

Rastreabilidade de requisitos

Independentemente do tipo da mudança, ela precisa ser gerenciada. Para uma efetiva gestão das mudanças, de acordo com Wiegers e Beatty (2013), a organização deve garantir que:

- as mudanças de requisitos propostas sejam avaliadas cuidadosamente antes de serem comprometidas;
- os indivíduos adequados tomem decisões de negócio embasadas sobre as mudanças solicitadas;
- a atividade de mudança se torne visível para as partes interessadas afetadas;
- as alterações aprovadas sejam comunicadas a todos os participantes afetados;
- o projeto incorpore mudanças de requisitos de forma consistente e eficaz.

Se uma mudança de requisitos for realizada sem os devidos cuidados, os impactos podem ser amplificados e tanto a qualidade quanto a satisfação do usuário podem ser comprometidas.

2 Elementos da matriz de rastreabilidade

Para que a análise de impacto de uma mudança possa ser realizada de forma adequada, é necessário garantir que as ligações entre os requisitos ou entre os requisitos e outros elementos do sistema sejam conhecidas e possam ser utilizadas como base para essa análise. A essa ligação damos o nome de **rastreabilidade de requisitos**.

Rastreabilidade se refere a rastrear um requisito à sua origem, o que é chamado de **rastreabilidade para trás** (*backward traceability*) e também aos seus desdobramentos, o que é chamado de **rastreabilidade para frente** (*forward traceability*). Essa é a chamada **rastreabilidade vertical** (BOURQUE; FAIRLEY, 2014).

Existe também a **rastreabilidade entre os requisitos** (POLH; RUPP, 2015), que mapeia as dependências entre os requisitos. Esta é chamada de **rastreabilidade horizontal**, conforme ilustra a Figura 2. A rastreabilidade é tipicamente implementada por meio de ferramentas de mapeamento de grafos ou matrizes bidimensionais (BOURQUE; FAIRLEY, 2014).

Fique atento

Rastreabilidade é definida pela norma internacional ISO/IEC/IEEE 12207:2017(E) como o "[. . .] grau de relacionamento que pode ser estabelecido entre duas ou mais entidades lógicas, especialmente entidades que têm um relacionamento predecessor-sucessor ou superior-subordinado entre si, como requisitos, elementos do sistema, verificações ou tarefas" (ISO/IEC/IEEE, 2017, p. 10, tradução nossa).

O SWEBOK v3 define rastreabilidade da seguinte forma: "O rastreamento de requisitos diz respeito à recuperação da fonte dos requisitos e à previsão dos efeitos dos requisitos" (BOURQUE; FAIRLEY, 2014, documento *on-line*, tradução nossa).

O CMMI-DEV v1.3 define a rastreabilidade bidirecional como sendo "Uma associação entre duas ou mais entidades lógicas que é detectável em qualquer direção" (CMMI PRODUCT TEAM, 2010, documento *on-line*, tradução nossa).

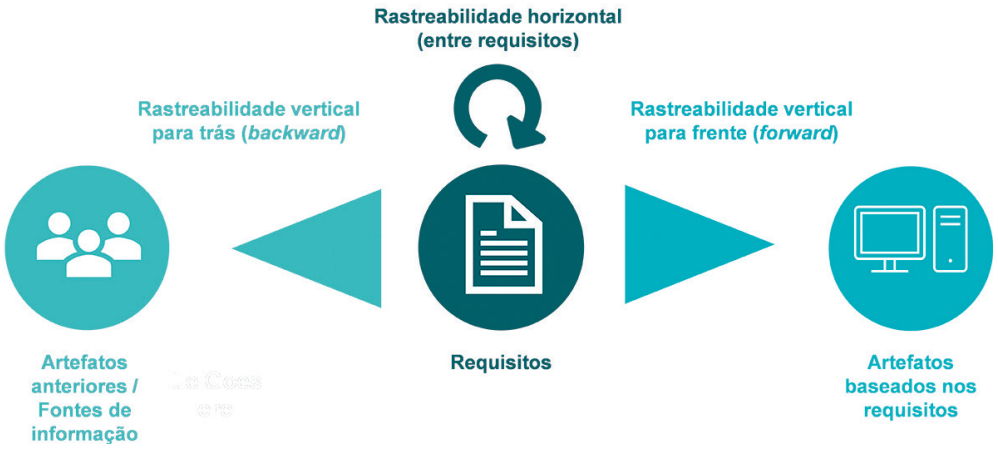


Figura 2. Tipos de rastreabilidade.

Importância da rastreabilidade

Conforme discutimos na seção anterior, a rastreabilidade de requisitos é muito útil para a análise do impacto das solicitações de mudança, mas esse não é o único benefício em sua utilização. De acordo com Wiegers e Beatty (2013, p. 494) a rastreabilidade nos permite o seguinte:

- **Encontrar requisitos faltantes:** buscar requisitos de negócios que não são rastreados para os requisitos de usuário, e requisitos de usuário que não são rastreados para nenhum requisito funcional. Isso permite identificar se existem pontos que não foram considerados ao longo do desdobramento dos requisitos.
- **Encontrar requisitos desnecessários:** procurar por quaisquer requisitos funcionais que não são rastreados de volta para requisitos de usuário e requisitos de negócios e que podem não ser necessários. Pohl e Rupp (2015) se referem a este aspecto como a detecção do *gold plating* ("banhado a ouro"), uma prática nociva ao projeto de implementar itens que não foram solicitados por nenhum dos *stakeholders*.
- **Realizar a certificação e conformidade:** você pode usar informações de rastreabilidade quando for certificar um produto crítico para segurança, para demonstrar que todos os requisitos foram implementados, embora isso não confirme que eles estão implementados corretamente. Informações rastreadas demonstram que os requisitos referentes a demandas por conformidade com regulamentações tenham sido incluídos e tratados, como é normalmente necessário para aplicações para empresas de saúde e financeiras.
- **Realizar a análise de impacto de mudanças:** sem informações de rastreamento, há uma boa chance de você ignorar um elemento do sistema que seria afetado se você adicionasse, apagasse ou modificasse um requisito em particular. Pohl e Rupp (2015) ainda complementam ressaltando a importância de identificar quais artefatos devem ser alterados em função da solicitação de mudança de um requisito.
- **Manter o *software*:** informações de rastreamento confiáveis facilitam a sua capacidade de fazer mudanças de forma correta e completa durante a manutenção. Quando as políticas corporativas ou regulamentações governamentais mudam, os sistemas de *software* muitas vezes precisam ser atualizados. Uma tabela que mostra onde cada regra de negócio aplicável foi abordada nos requisitos funcionais, projetos e código facilita a realização das mudanças necessárias de forma adequada. Pohl e Rupp (2105) complementam afirmando que é possível identificar relações de causa e efeito de uma falha, por exemplo, identificando os componentes que precisam ser alterados e apoiando as estimativas para a mudança.

- **Acompanhar o projeto:** se você registrar os dados de rastreamento durante o desenvolvimento, terá um registro preciso do *status* de implementação da funcionalidade planejada. A ausência de *links* indica produtos de trabalho que ainda não foram criados. Entenda aqui *link* como a ligação entre os elementos da matriz.
- **Realizar a reengenharia:** você pode listar as funções em um sistema existente que está substituindo e rastreá-las até onde elas são endereçadas nos requisitos do novo sistema e componentes de *software*.
- **Promover a reutilização:** as informações de rastreabilidade de reuso facilitam a reutilização dos componentes do produto, identificando pacotes de requisitos, projetos (*design*), códigos e testes relacionados.
- **Executar testes:** quando um teste falha, as ligações entre os testes, os requisitos e o código apontam os desenvolvedores para as áreas prováveis a serem examinadas para o defeito. A rastreabilidade apoia, portanto, as correções de defeitos encontrados em testes.

Além dos itens identificados por Wiegers e Beatty (2013), Pohl e Rupp (2015) ressaltam um aspecto adicional sobre a importância da rastreabilidade. Na visão dos autores, a rastreabilidade permite ainda realizar o seguinte:

- **Responsabilização:** a rastreabilidade de requisitos permite atribuir os esforços de desenvolvimento ao requisito. Depois que o requisito está implementado, por exemplo, todos os esforços aos artefatos associados podem ser sumarizados e relacionados ao requisito de origem.

Escolha do que será rastreado

Todos sabemos que manter atributos que permitam a rastreabilidade de todos os elementos de um projeto é uma tarefa praticamente impossível. São necessários muito esforço e muita disciplina para que os elementos selecionados sejam mantidos atualizados. Pior do que não haver um mecanismo de rastreabilidade é ter um mecanismo desatualizado e não confiável.

Se uma análise de impacto foi baseada em registros de rastreabilidade não confiáveis, corremos o risco de iniciar uma mudança sem a real visão sobre sua abrangência, o que pode levar a uma estimativa imprecisa de custos, cronograma e riscos.

Por outro lado, registrar todos os relacionamentos entre todos os elementos do desenvolvimento de *software* também é uma tarefa difícil e onerosa. Pode ser que haja relacionamentos muito triviais, que nunca serão utilizados e que, se tivessem sido analisados previamente, talvez não precisassem ser mantidos.

Apenas as informações com importância crítica para o desenvolvimento e para a manutenção do *software* devem ser registradas e mantidas (POLH; RUPP, 2015). Essa identificação dos itens que serão rastreados envolve uma tomada de decisão por parte do time de desenvolvimento, gestores e demais *stakeholders*. Ela é semelhante à decisão acerca da gerência de configuração, ou seja, que elementos devemos colocar sob controle de configuração mais rigoroso (com controle de *baseline*, por exemplo) e quais elementos podemos apenas versionar. É uma questão de custo × benefício.

São candidatos a serem itens rastreáveis:

- fontes de informação (origem do requisito);
- requisitos críticos para o negócio;
- requisitos que mais impactam na arquitetura;
- requisitos mais sujeitos a mudanças (maior volatilidade);
- requisitos mais críticos sob a ótica do teste (mais difíceis de testar, por exemplo);
- códigos críticos para o sistema;
- casos de teste críticos para o sistema.

3 Relacionamentos dos elementos da matriz de rastreabilidade

Nesta seção vamos apresentar a matriz de rastreabilidade e os relacionamentos entre os elementos que compõem a matriz. Melhor do que uma explicação teórica é vermos um exemplo prático de funcionamento.

A Figura 3 ilustra um recorte de um Sistema de Vendas. Esta é apenas uma representação esquemática para facilitar o entendimento dos conceitos envolvidos. Geralmente não usamos uma figura para representar essa rastreabilidade, pois é difícil de gerar e de manter.

O foco está no requisito funcional RF20, que se refere à necessidade do Pedido ser realizado por meio de comando de voz. Na figura é possível observar que existe uma **rastreabilidade para trás (*backward*)** referente ao *stakeholder* que realizou essa solicitação, que foi o Gerente de vendas. Pode-se notar a motivação do Gerente de vendas, que se relaciona com a simplicidade da

entrada de dados do pedido. Temos também a ligação com o objetivo estratégico do negócio OE3, que se refere a melhorar a produtividade da equipe de vendas em 10%. Vemos que essa informação é proveniente do planejamento estratégico da empresa.

Do lado direito da figura vemos a **rastreabilidade para frente (*forward*)**, ou seja, as ligações com os elementos que foram elaborados a partir desse requisito. Vemos aqui representados os Casos de Teste, em especial o Caso de Teste CT20.1, que se relaciona diretamente com o R20. Vemos também o Diagrama de Casos de Uso do sistema, com destaque para o caso de uso UC12 Entrar com pedido. Em seguida temos representado o código no qual esse requisito está implementado, no caso o módulo M1.

No centro da Figura 3 temos a representação da **rastreabilidade horizontal** em relação ao requisito não funcional RNF3, que estabelece que o pedido deverá ser executado em até 3 minutos. Lembre-se de que chamamos de rastreabilidade horizontal aquela que é feita entre elementos do mesmo tipo, ou seja, neste caso, entre os requisitos (funcional RF20 e não funcional RNF3).

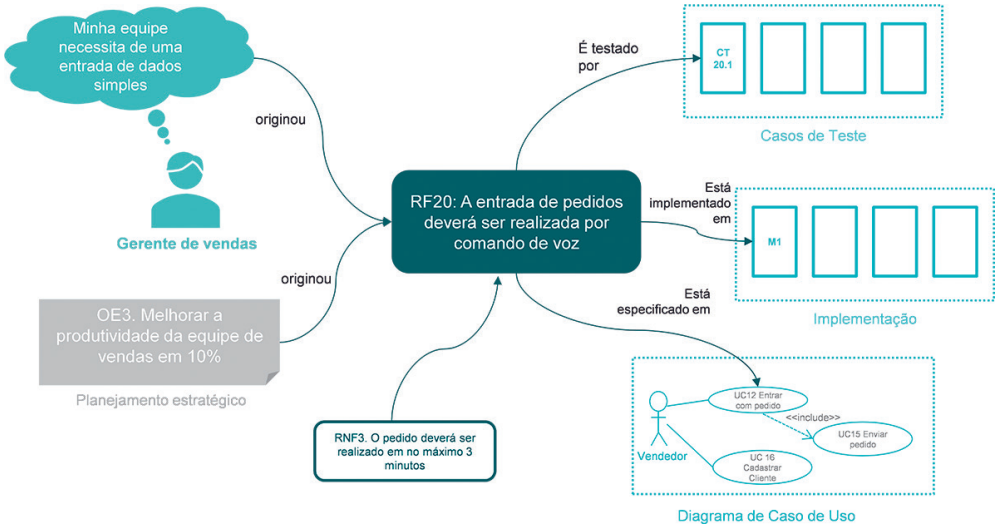


Figura 3. Exemplo de relacionamento entre os elementos de um produto de *software*.

Embora visualmente seja muito fácil de enxergar os relacionamentos que ligam os diversos elementos, na prática é inviável trabalhar com esse tipo de representação, a menos que seja implementado por ferramenta. Estamos utilizando esse formato apenas para mostrar o contexto e facilitar a compreensão dos elementos envolvidos.

A representação mais comum é no formato de matriz, que pode ser implementada usando-se planilha eletrônica ou ferramenta automatizada. Uma implementação eficiente de uma rastreabilidade mais robusta deve ser realizada por meio de ferramentas automatizadas. Existem ferramentas de modelagem UML, por exemplo, que já oferecem formas de registrar e manter a rastreabilidade dos requisitos e dos demais artefatos do ciclo de desenvolvimento de *software*. Pohl e Rupp (2015) sugerem a matriz de rastreabilidade ou os grafos de rastreabilidade.

Voltando para o exemplo da Figura 3, poderíamos implementar os elementos e seus relacionamentos em uma matriz, conforme representado na Figura 4, mais adiante. Vamos começar explicando as convenções utilizadas:

- células preenchidas em cinza não serão preenchidas;
- células em verde destacam as interseções com o requisito de interesse (RF20);
- células que contém um "X" denotam um relacionamento entre os elementos correspondentes.

Na primeira coluna listamos todos os elementos que poderão ser rastreados entre si. Nesse exemplo, listamos os *stakeholders*, os objetivos estratégicos, os requisitos funcionais, os requisitos não funcionais, os casos de uso, os casos de teste e os módulos de código.

Na primeira linha listamos os mesmos elementos. Note que não foram listados na linha 1 os *stakeholders* e nem os objetivos estratégicos, pois não temos interesse em rastrear o relacionamento entre eles, mas sim com os requisitos que eles geraram (funcionais e não funcionais).

Poderíamos ter mapeado todos os relacionamentos do Gerente de vendas com os elementos que foram desdobrados a partir do RF20, mas isso iria poluir desnecessariamente a planilha. Então assumimos que apenas nos interessa mapear quais foram os requisitos solicitados por cada *stakeholder*, ou seja, o único mapeamento possível na nossa planilha é entre o *stakeholder* solicitante e os requisitos funcionais e não funcionais (únicas células com o fundo branco). Raciocínio similar foi utilizado para a representação dos objetivos estratégicos. Note que a interseção dos *stakeholders* e dos objetivos estratégicos com os elementos Casos de Uso, Casos de Teste e Módulos foram suprimidos (células em cinza).

Podemos observar que estão marcados com um "X" todos os relacionamentos entre os elementos. A parte inferior da tabela está em cinza, denotando que são relacionamentos já representados na parte superior.

Em destaque verde temos a representação dos relacionamentos existentes com o requisito funcional RF20. Vemos que ele foi solicitado pelo Gerente de vendas e que se relaciona com o objetivo estratégico OE3. Esta é a chamada rastreabilidade para trás, ou seja, para a origem do requisito. Logo em seguida temos a rastreabilidade para frente, ou seja, quais foram os artefatos que foram gerados a partir do requisito funcional RF20. Temos então o caso de uso UC12, o caso de teste CT20.1 e o módulo implementado M1.

Temos mais um relacionamento representado, que é o relacionamento com o requisito não funcional RNF3. Esta é a rastreabilidade horizontal entre elementos do mesmo tipo (no caso, requisitos).

	Requisitos funcionais			Requisitos não funcionais			Casos de Uso			Casos de Teste			Módulos		
	RF01	(...)	RF20	RNF1	RNF2	RNF3	UC1	(...)	UC12	CT1.1	(...)	CT20.1	M1	(...)	M54
Gerente de Vendas			X			X									
Diretor de Vendas	X														
Vendedor															
OE1	X														
OE2															
OE3			X												
RF01					X		X			X					X
(...)															
RF20						X			X			X	X		
RNF1															
RNF2															
RNF3															
UC1															
(...)															
UC12															
CT1.1															
(...)															
CT20.1															
M1															
(...)															
M54															

Figura 4. Exemplo de matriz de rastreabilidade de requisitos usando planilha eletrônica.

Wiegers e Beatty (2013) destacam que o fato de haver um relacionamento entre o requisito funcional e o caso de teste não quer dizer que o código passou pelo teste ou que foi aprovado pelo teste. Neste caso, apenas está sendo representado que o requisito será testado usando os casos de teste com os quais está relacionado.

Vamos lembrar que aqui fizemos um recorte bem simplificado para que fosse possível mostrar em uma única página a forma de representação. Na prática, haveria dezenas de casos de teste para testar esta funcionalidade, bem como diversos arquivos de código-fonte. Possivelmente, haveria também mais requisitos não funcionais, relacionados, por exemplo, à segurança, usabilidade, desempenho etc.



Saiba mais

No caso de ambientes ágeis, os elementos mais presentes são o *blacklog* e, consequentemente, os itens do *backlog*, as histórias do usuário, os critérios de aceitação, as *sprints* e o próprio código. É comum que os critérios de aceitação funcionem como especificação das regras de negócio e que deem origem aos casos de teste que geralmente são automatizados. Bastaria substituir os elementos representados por eles. Embora o *product owner* funcione como o representante dos usuários para a equipe de desenvolvimento, cada item do *backlog* deve ter a sua origem registrada, ou seja, quem solicitou aquele item.

É importante lembrarmos que a rastreabilidade dos requisitos vai garantir que (CMMI INSTITUTE, 2018):

- os requisitos foram totalmente tratados nos entregáveis para o cliente;
- os requisitos de baixo nível podem ser rastreados a uma fonte válida;
- os requisitos foram implementados;
- os requisitos selecionados foram verificados e validados;
- os relacionamentos com outras entidades, como produtos de trabalho intermediários e finais, podem:
  - incluir dependências nas quais a mudança em um requisito pode implicar na mudança em outros requisitos;
  - apoiar no projeto (*design*) e avaliação do impacto das mudanças;
  - afetar a avaliação do impacto das mudanças;
  - apoiar a mudança antecipada do escopo;
- o projeto (*design*) e outras documentações refletem os requisitos;
- os planos de teste e casos de teste refletem os requisitos.

Manter a rastreabilidade é uma ação importante e que gera valor para o *software*, especialmente na fase de manutenção. O importante é manter o equilíbrio do que é rastreado para poder garantir que as informações estejam sempre atualizadas. Pior do que não ter uma informação, nesse caso, é ter uma informação incompleta ou incorreta.



Referências

BOEING 737 Max Lion Air crash caused by series of failures. *BBC*, 25 out. 2019. Disponível em: https://www.bbc.com/news/business-50177788. Acesso em: 19 jun. 2020.

BOURQUE, P.; FAIRLEY, R. *SWEBOK*: guide to the software engineering body of knowledge: version 3. USA: IEEE Computer Society, 2014. Disponível em: https://www.computer.org/education/bodies-of-knowledge/software-engineering. Acesso em: 19 jun. 2020.

CMMI INSTITUTE. *CMMI model V2.0.* Pittsburgh: CMMI Institute, 2018.

CMMI PRDUCT TEAM. *CMMI® for Development, Version 1.3.* Hascom AFB: Software Engineering Institute, 2010. Disponível em: https://resources.sei.cmu.edu/asset_files/TechnicalReport/2010_005_001_15287.pdf. Acesso em: 19 jun. 2020.

ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017(E)*: systems and software engineering: software life cycle processes. Switzerland: ISO, 2017.

POHL, K.; RUPP, C. *Requirements engineering fundamentals:* a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMANN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Parte 2

Gerenciamento de Requisitos

O conteúdo deste livro
é disponibilizado
por SAGAH.

SAGAH
SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Gerenciamento de requisitos

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Gerenciar o *status* dos requisitos.
- Analisar o impacto das solicitações de mudanças em projetos de desenvolvimento de *software*.
- Aplicar mecanismos de controle de versão de requisitos.

Introdução

Os requisitos são a base para todo o desenvolvimento de *software*. Requisitos mal compreendidos, omitidos, mal definidos e mal especificados são uma ameaça à qualidade do produto de *software* e à satisfação dos *stakeholders*. Um requisito mal compreendido, cujo problema permaneça não detectado até a sua homologação, trará prejuízos para a organização desenvolvedora, pois é certo que implicará, no mínimo, em retrabalho — isso sem contar os prejuízos à imagem da organização, além de riscos que possam representar aos usuários do produto.

As atividades da engenharia de requisitos têm o objetivo de apoiar o ciclo de desenvolvimento de *software* em todos os aspectos relacionados com os requisitos, que vão desde as atividades de elicitação até as atividades de verificação e validação, incluindo o gerenciamento de requisitos. O gerenciamento de requisitos envolve estabelecer mecanismos que permitam acompanhar o *status* dos requisitos ao longo do processo de desenvolvimento, garantindo que as versões estejam sob controle. Além disso, visa a tratar as solicitações de mudanças que irão inevitavelmente acontecer ao longo da vida de um produto de *software*, provendo mecanismos de rastreabilidade que apoiam a tomada de decisão.

Neste capítulo você vai ler sobre como gerenciar o *status* dos requisitos. Vai também ver como analisar os impactos de uma solicitação de mudanças e como aplicar mecanismos para realizar o controle de versão dos requisitos.

1 Gerenciamento do *status* dos requisitos

Requisitos são elementos que têm um ciclo de vida bem definido. Eles nascem a partir das atividades de elicitação, são analisados, especificados, validados e então seguem para o desenvolvimento, em que serão codificados, testados e entregues na forma de um produto de *software*. A partir da sua entrega, o requisito passa a viver para sempre dentro do produto, até que alguma solicitação de mudança o elimine ou modifique.

Para que os requisitos permaneçam aderentes às necessidades dos *stakeholders* e não venham a ser uma fonte de inconsistência, retrabalho e riscos, eles precisam ser gerenciados ao longo de toda a sua vida, seja no ciclo de desenvolvimento, seja no ciclo de manutenção. A Figura 1 apresenta as atividades da engenharia de requisitos e o seu relacionamento com o desenvolvimento, mas podemos entender que isso se aplica também para o ciclo de vida da manutenção, ou seja, uma solicitação de mudança é recebida e as atividades de elicitação, análise, especificação e validação são realizadas, gerando uma nova versão do produto.

Na Figura 1 utilizamos um semicírculo para representar as atividades de engenharia de requisitos, para que não fosse denotado um ciclo de vida em cascata, mas sim atividades que precisam ser executadas antes que o requisito seja implementado, qualquer que seja o método utilizado. As atividades de gerenciamento de requisitos permearão toda a vida do produto de *software*, pois as manutenções realizadas no produto sempre estarão relacionadas aos requisitos, seja para alterá-los, seja para deletá-los ou para inserir novos requisitos.

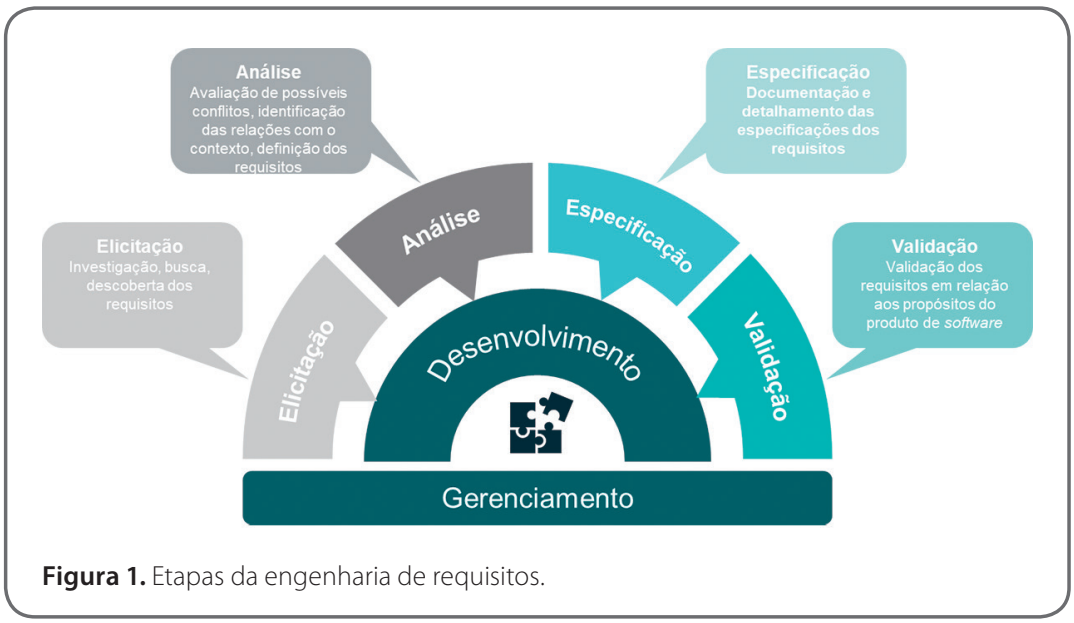


Figura 1. Etapas da engenharia de requisitos.

Podemos entender que o gerenciamento de requisitos "[…] inclui todas as atividades que mantêm a integridade, a acurácia e a atualidade dos acordos sobre os requisitos ao longo do projeto" (WIEGERS; BEATYY, 2013, p. 458, tradução nossa). Pohl e Rupp (2015, p. 111, tradução nossa) definem que:

> [...] o gerenciamento de requisitos compreende atribuir intencionalmente atributos aos requisitos, definindo visões sobre os requisitos, priorizando requisitos e rastreando os requisitos, bem como versionando os requisitos, gerenciando mudanças nos requisitos e medindo os requisitos.

A Figura 2 representa as categorias de atividades que são necessárias para a realização do gerenciamento dos requisitos na visão de Wiegers e Beatty (2013). São elas:

- **Controle de versão:** os requisitos precisam ter as suas versões controladas. Isso é realizado por meio da definição de um esquema para identificação das versões, bem como pelo seu acompanhamento durante todo o ciclo de vida dos requisitos e dos conjuntos de requisitos.
- **Controle de mudanças:** as mudanças nos requisitos precisam ser controladas, de modo a garantir a integridade dos elementos envolvidos nas mudanças.
- **Acompanhamento do *status*:** acompanhamento dos *status* dos requisitos, o que inclui a definição dos possíveis *status* a serem controlados.
- **Rastreabilidade:** envolve a possibilidade de rastrear os requisitos entre si e entre os demais elementos do sistema.

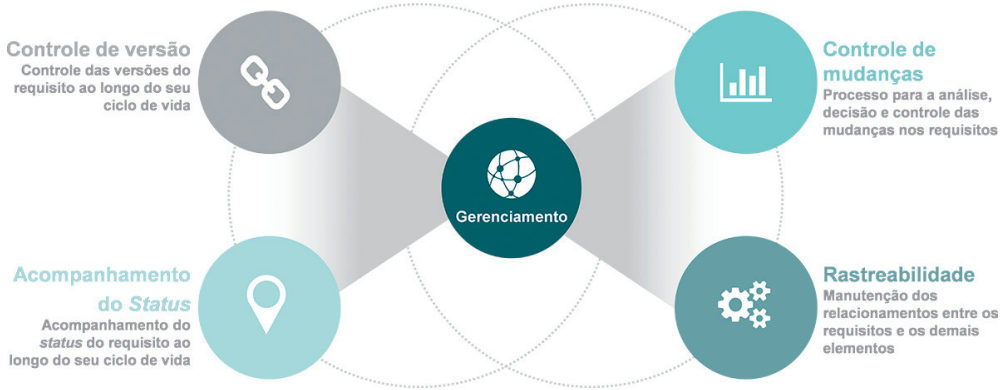


Figura 2. Categorias das atividades do gerenciamento de requisitos.

Fonte: Adaptada de Wiegers e Beatty (2013).

Para que as atividades de gerenciamento de requisitos sejam implementadas na organização, é importante que a gerência superior compreenda os benefícios do gerenciamento e que as pessoas encarregadas dessas atividades sejam treinadas adequadamente. Se um processo claro não for estabelecido, então, o gerenciamento dos requisitos não acontecerá.

Neste capítulo trataremos das atividades de controle de versão, controle de mudanças e acompanhamento do *status*. A rastreabilidade não será tratada no escopo deste capítulo. Esta sessão é dedicada ao acompanhamento do *status* do requisito.

Atributos dos requisitos

Os requisitos podem ser caracterizados por um conjunto de atributos, que ajudarão no seu gerenciamento e, especialmente, na sua priorização, conforme apresentado no Quadro 1. Esses atributos podem ser customizados de acordo com as características do projeto, o contexto da organização ou requisitos legais.

Quadro 1. Atributos de requisitos

Atributo	Significado
Identificador	Identificador único e curto do requisito
Nome	Nome único que caracteriza o requisito
Descrição	Descreve brevemente o requisito
Versão	Versão atual do requisito
Autor	Autor do requisito
Fonte	Fonte do requisito
Estabilidade	Especifica a estabilidade aproximada do requisito. A estabilidade é a quantidade de mudanças que são esperadas no requisito. Valores possíveis podem ser "fixo", "estabelecido", "volátil"
Risco	Especifica o risco baseado na estimativa da quantidade de perdas e danos e a probabilidade de ocorrência

(Continua)

(Continuação)

Quadro 1. Atributos de requisitos

Atributo	Significado
Prioridade	Prioridade do requisito em relação às propriedades escolhidas (por exemplo, "importância para aceitação no mercado", "custo da perda da oportunidade")
Responsável	Especifica a pessoa, grupo de *stakeholders*, organização ou unidade organizacional que é responsável pelo conteúdo do requisito
Tipo do requisito	Especifica o tipo do requisito (por exemplo, requisito funcional, requisito de qualidade ou restrição), dependendo do esquema de classificação aplicado
Status do conteúdo	Especifica o *status* atual do conteúdo do requisito (por exemplo, "ideia", "conceito", "conteúdo detalhado")
Status da validação	Especifica o *status* atual da validação do requisito (por exemplo, "não validado", "com erro", "em correção")
Status do acordo	Especifica o *status* atual da negociação (por exemplo, "não negociado", "negociado", "em conflito")
Esforço	Esforço estimado/real para implementar o requisito
Versão (*release*)	Designação da versão (*release*, *sprint*) na qual o requisito deverá ser implementado
Obrigação legal	Grau de obrigação legal do requisito
Referência cruzada	Especifica as relações com outros requisitos, por exemplo, se é sabido que a implementação do requisito requer que outro requisito esteja implementado
Observações gerais	Neste atributo, informações arbitrárias que são consideradas relevantes podem ser documentadas, por exemplo, se a negociação desse requisito está agendada para discussão durante a próxima reunião com os *stakeholders*

Fonte: Adaptado de Pohl e Rupp (2015).

Durante o gerenciamento do projeto e a priorização dos requisitos, os atributos dos requisitos podem ser usados individualmente ou combinados entre si. Por exemplo, se um requisito tiver uma prioridade alta sob a ótica do negócio, mas for um requisito instável, que ainda não está bem definido, sua implementação representará um risco também elevado, devido à sua instabilidade. Nesse caso, esforços deverão ser envidados para o esclarecimento desse requisito, de forma que possa ser priorizado para implementação. Se isso não for possível, então, a prioridade terá que ser revista, pois a implementação de um requisito instável certamente implicará em retrabalho, o que significa custos adicionais para o projeto.



Fique atento

Embora seja tentador definir diversos atributos para caracterizar um requisito, favorecendo o seu gerenciamento, isso pode custar caro em termos de manutenção. Para cada um dos atributos de um requisito, deverá ser prevista uma forma de **garantir a sua integridade** ao longo do tempo. Por exemplo, quando um requisito que, no primeiro momento, estava instável se tornar mais claro e estável, será necessário refletir essa mudança no atributo relativo à estabilidade. Isso vale para todos os demais atributos.

Dessa forma, é importante **selecionar aqueles mais relevantes** de acordo com o contexto do projeto, pois eles precisarão estar sempre atualizados para que possam ser úteis para o gerenciamento dos requisitos. Para selecionar os atributos que serão mantidos, imagine em que situações eles serão úteis. Pense que perguntas poderão ser respondidas por eles e com que frequência você se faz essas perguntas. É possível que em alguns momentos você pense: "seria tão legal ter este atributo!". Em seguida pense: "Se fosse eu que tivesse que manter atualizados todos estes atributos, eu continuaria investindo em mantê-los?". Se a resposta for sim, este é um candidato. Se for não, aproveite para trocar uma ideia com outros colegas, para analisar a real importância da manutenção dele.

Definição dos tipos de *status* que serão acompanhados

Inicialmente, precisamos definir que *status* do requisito desejamos acompanhar. Para escolher os estados que nos interessam, é preciso ter em mente que, da mesma forma que os atributos, manter o *status* atualizado é uma tarefa custosa. Todos os envolvidos devem se comprometer na atualização do *status* à medida que finalizam uma atividade.

É necessário realizar uma análise de custo-benefício. Um maior número de estados permite também uma maior precisão no acompanhamento, mas é mais difícil de manter atualizado. Um menor número, embora permita menor detalhamento no acompanhamento, será menos custoso e mais fácil de manter. Por esse motivo, o ideal é começar com uma pequena quantidade de estados a serem acompanhados e, posteriormente, introduzir novos, caso sejam necessários.

Embora seja possível realizar esse acompanhamento de forma manual, usando planilhas, por exemplo, essa não é a situação ideal. O melhor é utilizar uma ferramenta automatizada para apoiar esta tarefa. O Quadro 2 apresenta uma sugestão de *status* a serem monitorados.

Quadro 2. *Status* de requisitos

Status	**Definição**
Proposto	O requisito foi solicitado por uma fonte autorizada
Em progresso	Um analista de negócios está trabalhando ativamente para detalhar o requisito
Rascunhado	A versão inicial do requisito foi escrita
Aprovado	O requisito está alocado a uma *baseline* específica
Implementado	O requisito está implementado
Verificado	O requisito satisfez aos critérios de aceitação
Deferido	Um requisito está planejado para ser implementado em uma versão posterior
Deletado	Um requisito aprovado foi retirado da *baseline*
Rejeitado	O requisito foi processado, mas não há planejamento para a sua implementação

Fonte: Adaptado de Wiegers e Beatty (2013).



Fique atento

Os *status* a serem monitorados precisam ser definidos antes do início do projeto, pois todas as partes envolvidas precisam ter ciência do tipo e periodicidade com que os *status* deverão ser atualizados. Além do mais, se houver ferramentas envolvidas, elas precisarão ser configuradas para contemplar os *status* selecionados.

Etapas técnicas mais detalhadas podem também ser adicionadas. Cada vez que o requisito troca de mãos, pode acontecer uma mudança de *status*, e isso pode incluir especificações intermediárias (como, por exemplo, "requisito projetado"). Mas sempre tenha em mente a facilidade de manutenção do *status*.

Nos ambientes que utilizam métodos ágeis, normalmente o *status* de uma história de usuário é mantido durante a fase de implementação. É comum encontrar empresas que utilizam as histórias escritas em Post-it® e os quadros Kanban para acompanhar o *status* da história. No entanto, enquanto a história está no backlog do produto, seu status não é atualizado. Etapas intermediárias podem também ser consideradas, caso a organização utilize ferramentas automatizadas para implementar o quadro Kanban.



Saiba mais

Em Leffingwell (2011), é possível compreender como os requisitos são tratados no *backlog* do produto e como acompanhar a velocidade com que eles são retirados do *backlog* e entregues ao cliente implementados. Wiegers e Beatty (2013) apontam que é possível adotar o mesmo raciocínio para os ambientes ágeis, definindo os *status* para as histórias do usuário, acompanhe:

- No *backlog* — a história ainda não está alocada a uma iteração.
- Definida — detalhes da história já foram discutidos e compreendidos, e os critérios de aceitação já foram escritos.
- Em progresso — a história está sendo implementada.
- Completada — a história já foi completamente implementada.
- Aceita — os testes dos critérios de aceitação passaram.
- Bloqueada — o desenvolvedor está incapacitado de prosseguir até que alguma coisa a mais esteja resolvida.

Acompanhamento do *status* dos requisitos

Quando se combinam os atributos dos requisitos com o seu *status*, é possível responder a questões relativas ao acompanhamento do projeto, de uma iteração ou do próprio produto. Isso é particularmente útil quando temos projetos maiores ou equipes globalmente distribuídas.

Por exemplo, em um dado momento, é possível determinar quais são os requisitos que estão alocados a um desenvolvedor específico e em que situação eles estão. Pode-se ainda querer saber quais requisitos ainda não foram validados pelo solicitante e já estão alocados a uma versão ou *sprint*. Enfim, as possibilidades são inúmeras e permitem que a capacidade de acompanhamento e de tomada de decisão sejam ampliadas.

Ao percebermos que determinado atributo ou *status* se tornou muito custoso para ser mantido, a reflexão que deve ser feita é: vale a pena manter este atributo ou este *status*? Ele ainda é usado para responder a alguma questão técnica ou de gestão? Se houver dúvida, é melhor não utilizar mais o atributo ou *status*.

Para garantir que as informações são consistentes e estão sendo mantidas atualizadas, o ideal é promover **auditorias por amostragem** de tempos em tempos. Isso vai garantir que as informações que estão armazenadas são confiáveis, atualizadas e que seguem as políticas definidas. Isso aumentará o grau de confiança das pessoas em que esses parâmetros podem apoiar a tomada de decisão efetivamente.

É muito importante manter o registro de questões que ficam pendentes acerca dos requisitos, como, por exemplo, se determinado requisito permanece no *status* de "pendente de definições", mais conhecido pela sua sigla em inglês TBD (*to be defined*). A equipe deve manter essa lista de pendências sempre atualizada, preferencialmente em uma ferramenta de acompanhamento de pendências (*issue tracker*). Em projetos grandes, pendências não registradas podem ser esquecidas pela equipe.

2 Impacto das solicitações de mudança

A segunda atividade envolvida no gerenciamento de requisitos, conforme ilustrado na Figura 2, é o controle de mudanças, ou seja, como serão tratados os pedidos de mudança. As mudanças irão acontecer, de forma inevitável, uma vez que os produtos de *software* são elementos vivos, que estão situados em um contexto cuja velocidade de mudança cresce dia a dia. Chemuturi (2013, tradução nossa) aponta as seguintes causas para a mudança de requisitos:

- o ambiente no qual a organização opera sofreu uma mudança à qual a organização precisa responder;
- a gerência da organização pode realizar uma mudança na sua operação e isso pode levar a mudanças nas funcionalidades essenciais dos requisitos;
- alguns dos usuários finais podem ter esquecido de alguns requisitos e lembrado de novos requisitos depois que os requisitos foram congelados; isso irá demandar uma mudança nos requisitos;
- uma nova lei, regulamentação ou ato governamental pode causar mudanças depois que os requisitos foram congelados;
- atividades de melhoria de processos podem fazer com que algumas práticas de negócio sejam modificadas, e isso pode requerer mudanças nos requisitos;
- a análise dos dados de projetos encerrados pode revelar algumas anomalias que podem resultar em mudanças nos requisitos.

Essas são mudanças de ordem mais abrangente e abrigam em si diversas subcategorias; no entanto, Chemuturi (2013, tradução nossa) aponta outras questões de ordem mais técnica, que podem também levar a mudanças nos requisitos, são elas:

- durante a etapa de projeto (*design*) ou a etapa de construção, pode ser descoberto que a implementação de alguns requisitos já congelados pode não ser possível devido a razões técnicas ou considerações sobre custos; isso pode levar à necessidade de mudanças nos requisitos;
- alguns sistemas de *software* podem liberar *patches* ou *service packs* que afetam o projeto (*design*) do *software*, causando mudanças nos requisitos;
- uma nova ameaça ou brecha de segurança pode ter sido descoberta no sistema de *software*, necessitando de uma revisão dos requisitos;
- alguém pode ter descoberto uma forma melhor de atender à funcionalidade, causando mudanças nos requisitos.

Para ser possível gerenciar as mudanças nos requisitos, um instrumento é muito importante: a **rastreabilidade**.



Fique atento

Rastreabilidade é definida pela norma internacional ISO/IEC/IEEE 12207:2017(E), como o "[...] grau de relacionamento que pode ser estabelecido entre duas ou mais entidades lógicas, especialmente entidades que têm um relacionamento predecessor-sucessor ou superior-subordinado entre si, como requisitos, elementos do sistema, verificações ou tarefas" (ISO/IEC/IEEE, 2017, p. 10, tradução nossa).

O SWEBOK v3 define rastreabilidade da seguinte forma: "O rastreamento de requisitos diz respeito à recuperação da fonte dos requisitos e à previsão dos efeitos dos requisitos" (BOURQUE; FAIRLEY, 2014, documento *on-line*, tradução nossa).

O CMMI-DEV v1.3 define a rastreabilidade bidirecional como sendo "Uma associação entre duas ou mais entidades lógicas que é detectável em qualquer direção" (CMMI PRODUCT TEAM, 2010, documento *on-line*, tradução nossa).

Da mesma forma que discutimos, na seção anterior, sobre o custo-benefício de se manter este ou aquele atributo do requisito e este ou aquele tipo de *status*, devemos também definir que ligações de rastreabilidade manteremos, uma vez que é também oneroso mantê-las. Minimamente, deve-se ter o controle dos relacionamentos entre os seguintes elementos:

- fontes de informação (origem do requisito);
- requisitos críticos para o negócio;
- requisitos que mais impactam na arquitetura;
- requisitos mais sujeitos a mudanças (maior volatilidade);
- requisitos mais críticos sob a ótica do teste (mais difíceis de testar, por exemplo);
- códigos críticos para o sistema;
- casos de teste críticos para o sistema.

Aceitar a mudança como parte fundamental da vida do *software* é inevitável e inquestionável. Isso não quer dizer que a equipe de desenvolvimento tenha que dizer sim para todos os pedidos que recebe, mas sim que um processo para realizar o gerenciamento da mudança deve ser estabelecido, mantido e seguido por todos os envolvidos.

Processo da solicitação de mudança

Quando falamos em processo definido, é comum que as pessoas pensem em burocracia e entraves para as mudanças. A ideia aqui não é essa. O que é necessário é que os procedimentos para mudar um requisito sejam claros e visíveis a todos os *stakeholders* e que as mudanças aprovadas estejam alinhadas às expectativas de todos os envolvidos. Além disso, é necessário que as mudanças sejam comunicadas para todos os que possam vir a ser afetados por ela, o que inclui os usuários em geral, a equipe de desenvolvimento, os envolvidos na homologação e a equipe de operações.

A Figura 3 ilustra as mudanças de *status* da solicitação de mudança. A partir dela é possível inferir o processo necessário para implementar o controle de mudanças.

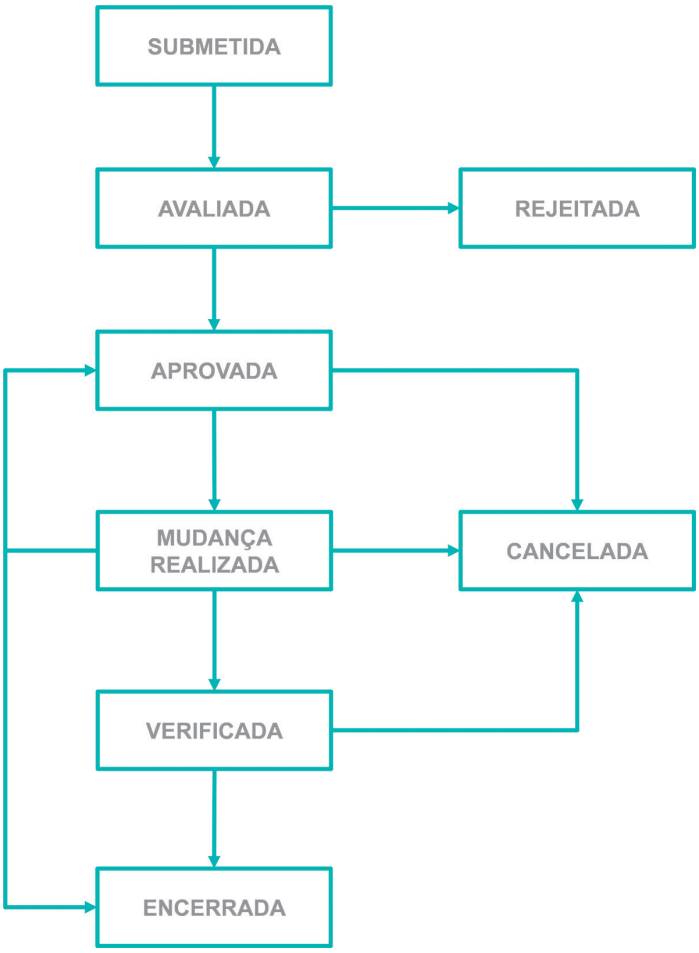


Figura 3. *Status* da solicitação de mudanças.

Fonte: Adaptada de Wiegers e Beatty (2013).

Como você pode observar no diagrama de blocos da Figura 3, o ponto de entrada é uma solicitação de mudança **enviada** ou **submetida** por algum dos *stakeholders* qualificados, ou seja, aqueles que podem submeter solicitações de mudanças. Uma solicitação de mudança pode ser identificada por diversas fontes (CHEMUTURI, 2013): clientes, usuários finais, equipe do projeto, equipe de testes, equipe de processos etc.

Em seguida, ela é **avaliada**. Se for considerada improcedente, será **rejeitada**. Se for considerada procedente, ela segue em frente no fluxo e passa a ser considerada **aprovada.**

Quem avalia se uma solicitação de mudança pode prosseguir é o Comitê de Controle de Mudanças (CCM, ou, em inglês, CCB — *change control board*). Um CCM pode ser uma equipe que toma as decisões ou um único indivíduo. Quanto maior e mais crítico o projeto, mais formal é o CCM. Embora possa parecer que o CCM seja uma burocracia desnecessária, pense que ele é um elemento fundamental para melhorar a qualidade das decisões. Em projetos ou produtos muito grandes, o CCM pode ser composto por diversos membros, e cada membro pode analisar aspectos específicos. Enquanto alguns podem estar mais atentos a aspectos de negócio, outros se dedicarão aos aspectos técnicos.



Saiba mais

A composição do CCM pode variar de acordo com o contexto. Pohl e Rupp (2011, tradução nossa) sugerem os seguintes papéis:

- gerente de mudanças;
- contratante;
- arquiteto;
- desenvolvedor;
- gerente de configuração;
- representante do cliente;
- gerente do produto;
- gerente do projeto;
- representante da garantia da qualidade;
- engenheiro de requisitos.

Wiegers e Beatty (2013) ainda sugerem que sejam levados em consideração os papeis relacionados ao *marketing*, além das equipes de suporte técnico e *help desk*.

Em ambientes ágeis, na maioria das vezes, as decisões são tomadas pelo *product owner* (PO). Mesmo nesses casos, vale lembrar que o PO é um representante dos interesses de outros *stakeholders* e que, portanto, deverá envolver outras pessoas na tomada de decisão. Nos métodos ágeis é comum que as mudanças possam ser realizadas sempre a partir da próxima *sprint,* uma vez que a *sprint* em andamento não pode ser alterada. Na prática, há momentos em que isso não é possível e o escopo de uma *sprint* em andamento precisa ser modificado.

Mais um ponto importante a ser lembrado diz respeito ao registro das solicitações de mudanças e das decisões tomadas, uma vez que as mudanças precisarão ser comunicadas e acessadas sempre que necessário. Ferramentas que apoiam equipes de *help desk* podem ajudar, uma vez que são baseadas em *workflows* de aprovação e trabalham com a gerência dos *status* das solicitações.

Uma vez aprovada a mudança, ela é atribuída a um indivíduo ou equipe, que fará o desenvolvimento da solução. Essa mudança poderá ser acatada seguindo o fluxo normal da equipe de desenvolvimento, ou pode se configurar como uma solicitação urgente que precisa de tratamento imediato. Independentemente do momento e do tratamento, é importante que todos os elementos afetados sejam mantidos consistentes entre si. Isso quer dizer que, quando um requisito é alterado, todos os artefatos relacionados precisam ser revistos e alterados. Isso inclui alterar as informações de rastreabilidade, sejam elas mantidas por ferramentas, sejam na forma de planilhas eletrônicas.

Quando a solicitação de mudança estiver na situação de **mudança realizada**, ela pode ser testada e então passa a ocupar o *status* de **verificada**. Uma vez implantada, ela passa para o estado de **encerrada.**

Note no fluxo da Figura 3 que a solicitação de mudança pode ser **cancelada** em qualquer ponto do ciclo, o que significa que ela não mais será implementada ou liberada. Nos casos em que a mudança para a situação de cancelada se der após ter sido realizada a implementação, então o que foi mudado deve ser retornado para a sua situação anterior.

As situações possíveis de saída desse processo são os seguintes *status*: a mudança ter sido **rejeitada**; a mudança ter sido **cancelada**; ou a mudança ter sido **encerrada**. Nesse último caso, significa ser encerrada com sucesso e a mudança implementada.

Nesse processo, o ponto crítico está na avaliação da solicitação de mudança, pois ela implica em uma análise aprofundada dos impactos da mudança. Isso inclui visitar a matriz de rastreabilidade em busca de elementos que serão potencialmente impactados pela mudança. Essa análise pode ser muito simples, mas pode também ser complexa e englobar diversos *stakeholders* e procedimentos formais de análise.



Saiba mais

Se você quiser se aprofundar sobre os critérios de análise das solicitações de mudanças, consulte o livro *Software Requirements*, de Wiegers e Beatty (2013), focando especial atenção à seção "Change impact analysis" (Análise do impacto da mudança).

3 Mecanismos de controle de versão de requisitos

Em todos os momentos do ciclo de desenvolvimento de *software*, a equipe deve ter acesso ao conjunto de requisitos, que precisa estar atualizado e aprovado para a implementação. Caso os requisitos não estejam atualizados, as pessoas tomarão decisões em cima de dados incorretos, o que poderá levar a riscos para o projeto e para o produto.

O controle de versões dos requisitos, ilustrado como um dos quatro pilares do gerenciamento de requisitos na Figura 2, engloba as seguintes atividades:

- definir um esquema de identificação de versões;
- rastrear versões individuais dos requisitos;
- rastrear versões do conjunto de requisitos.

Baseline de requisitos

As atividades de desenvolvimento dos requisitos (elicitação, análise, especificação e verificação) envolvem a produção de diversos artefatos, cada um deles atendendo aos seus propósitos. Vamos analisar o caso de uma empresa que utilize como artefatos de requisitos a lista de requisitos funcionais e não funcionais, o diagrama de casos de uso, as especificações de caso de uso, as especificações de requisitos não funcionais e os casos de teste.

Quando a atividade de elicitação de requisitos acontecer, ela irá gerar a lista de requisitos, que poderá ser armazenada tanto no formato de uma tabela quanto em uma ferramenta de gerenciamento de requisitos. Durante as atividades de análise, serão produzidos os diagramas de caso de uso. Estes serão gerados e armazenados em ferramentas de modelagem UML. Nas atividades de especificação, serão geradas as especificações de casos de uso e dos requisitos não funcionais. Isso pode ocorrer em ferramentas de modelagem que tenham mais recursos, ou pode acontecer em documentos produzidos em processador de texto. Na atividade de verificação, serão gerados e executados os casos de teste. Estes podem ser definidos em diversos formatos diferentes, incluindo testes automatizados.

Quando um conjunto de requisitos é acordado entre a equipe de desenvolvimento e os *stakeholders,* então, ele passa a constituir o que denominamos ***baseline* de requisitos**. Uma *baseline* pode ser compreendida como uma versão estável dos requisitos, que será usada como base para gerar os demais artefatos do projeto, incluindo o código. Quando um conjunto de requisitos é acordado, ele será a base para gerar as especificações detalhadas, as interfaces, os casos de teste e o próprio código.

Uma *baseline* precisa ser armazenada, controlada e comunicada a todos os envolvidos. Quando um desenvolvedor inicia a codificação, ele tem que ter confiança de que está se baseando nas especificações corretas, caso contrário o trabalho dele será desperdiçado. Da mesma forma, o analista de testes irá gerar o conjunto de casos de teste que serão utilizados para validar os requisitos, e, se eles se basearem em requisitos incorretos, também haverá retrabalho nesta etapa.

Quando uma solicitação de mudança for aprovada pelo CCM, todos os artefatos afetados (identificados por meio da rastreabilidade de requisitos) deverá ser alterado de modo a permanecerem alinhados e consistentes. Neste momento, todas as atividades em andamento precisam também ser alinhadas. Suponha que o requisito funcional R15 está sendo alterado por uma solicitação de mudança e, de acordo com seu controle de *status,* ele já está na etapa de implementação e todos os casos de teste referentes a ele já estão definidos. Quando essa solicitação de mudança for aprovada, será necessário revisar a *baseline* de requisitos e todos os artefatos associados. Os envolvidos no desenvolvimento também terão que ser envolvidos para revisarem suas atividades, pois elas podem estar sendo afetadas pela mudança na *baseline.*

O nível de controle de cada um dos artefatos deverá ser analisado de acordo com a criticidade e os impactos para o projeto. Cada organização define os procedimentos de controle de versões mais apropriados à realidade dos projetos que ela desenvolve.



Referências

BOURQUE, P.; FAIRLEY, R. *SWEBOK*: guide to the software engineering body of knowledge: version 3. USA: IEEE Computer Society, 2014. Disponível em: https://www.computer.org/education/bodies-of-knowledge/software-engineering. Acesso em: 22 jun. 2020.

CHEMUTURI, M. *Requirements engineering and management for software development projects*. New York: Springer, 2013.

CMMI INSTITUTE. *CMMI model V2.0.* Pittsburgh: CMMI Institute, 2018.

CMMI PRODUCT TEAM. *CMMI® for Development, Version 1.3.* Hascom AFB: Software Engineering Institute, 2010. Disponível em: https://resources.sei.cmu.edu/asset_files/TechnicalReport/2010_005_001_15287.pdf. Acesso em: 22 jun. 2020.

ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017(E)*: systems and software engineering: software life cycle processes. Switzerland: ISO, 2017.

LEFFINGWELL, D. *Agile software requirements*: lean requirements practices for teams, programs, and enterprises. Upper Saddle River: Adisson-Wesley, 2011.

POHL, K.; RUPP, C. *Requirements engineering fundamentals:* a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

PRESSMANN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Parte 3

Especificação de Requisitos Para um Cenário de Negócios

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Especificação de requisitos para um cenário de negócios

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Analisar os elementos de um cenário de negócios.
- Selecionar a técnica mais adequada para a elicitação de requisitos de um cenário de negócios.
- Especificar o escopo completo dos requisitos de *software* em um cenário de negócios.

Introdução

É inquestionável o papel dos *smartphones* no dia a dia das pessoas, seja para atividades de lazer, seja para atividades profissionais. Com o desenvolvimento da tecnologia, maior capacidade de processamento e armazenamento, os *smartphones* se tornaram uma fonte para resolução de diversos problemas do dia a dia, e passamos a usá-los para nos comunicar, tirar fotos e fazer vídeos, acessar as redes sociais, ler *e-mails*, fazer transações bancárias, ler livros, fazer reuniões remotas, pedir comida e muito mais.

Entre as diversas funções do *smartphone* estão aquelas relacionadas aos serviços públicos, cujos aplicativos nos permitem consultar o título de eleitor digital, a carteira de motorista digital, INSS, ENEM e diversos outros serviços. Este capítulo vai focar o desenvolvimento de um aplicativo para apoiar o Detran (Departamento de Trânsito) na comunicação com seus usuários, os proprietários e condutores de veículos automotores. O objetivo é percorrer as atividades da engenharia de requisitos, uma das partes fundamentais para a qualidade dos produtos de *software*, apresentando os conceitos associados de forma prática.

A engenharia de requisitos está organizada em dois grandes grupos de atividades: o desenvolvimento dos requisitos e o gerenciamento dos requisitos. No desenvolvimento dos requisitos estão concentradas as atividades relacionadas à compreensão do contexto do projeto, elicitação, análise, especificação e validação dos requisitos. Já no gerenciamento dos requisitos estão concentradas as atividades relacionadas com a rastreabilidade, o acompanhamento do *status*, o controle de versão e o controle de mudanças nos requisitos.

Neste capítulo vamos nos concentrar nas atividades relacionadas com o desenvolvimento de requisitos, focando em um caso prático de aplicação das técnicas da engenharia de requisitos. Nele você vai ver como analisar um cenário de negócios, em busca da compreensão do contexto do projeto e da identificação das fontes de informação. Vai também ver como poderá selecionar as melhores técnicas de elicitação de requisitos em um cenário de aplicação. Por fim, vai ler sobre como especificar os requisitos que atendem a um escopo completo de um projeto de *software*.

1 Análise do cenário de negócios

A engenharia de requisitos é uma peça fundamental na busca pela qualidade no desenvolvimento de *software*. De nada adianta termos excelentes desenvolvedores se eles estão construindo certo o produto errado. Requisitos mal compreendidos, mal definidos e mal especificados trarão consequências desastrosas para o projeto. Na melhor das hipóteses haverá retrabalho e insatisfação do cliente. Em situações mais graves, poderá haver prejuízos financeiros ou até mesmo a perda de vidas.

A Figura 1 apresenta as atividades envolvidas no desenvolvimento dos requisitos. Embora possa, no primeiro momento, parecer que se trata de atividades sequenciais, que se desenrolam no formato cascata, este não é o caso. Na verdade, trata-se de atividades de natureza iterativa e incremental, que podem ocorrer diversas vezes ao longo do desenvolvimento de um projeto.

Também não estamos aqui limitando a interpretação a uma forma tradicional de gerenciamento, ou seja, as atividades relacionadas a requisitos terão que acontecer, seja em um ambiente de gestão mais tradicional, seja em um ambiente mais ágil.

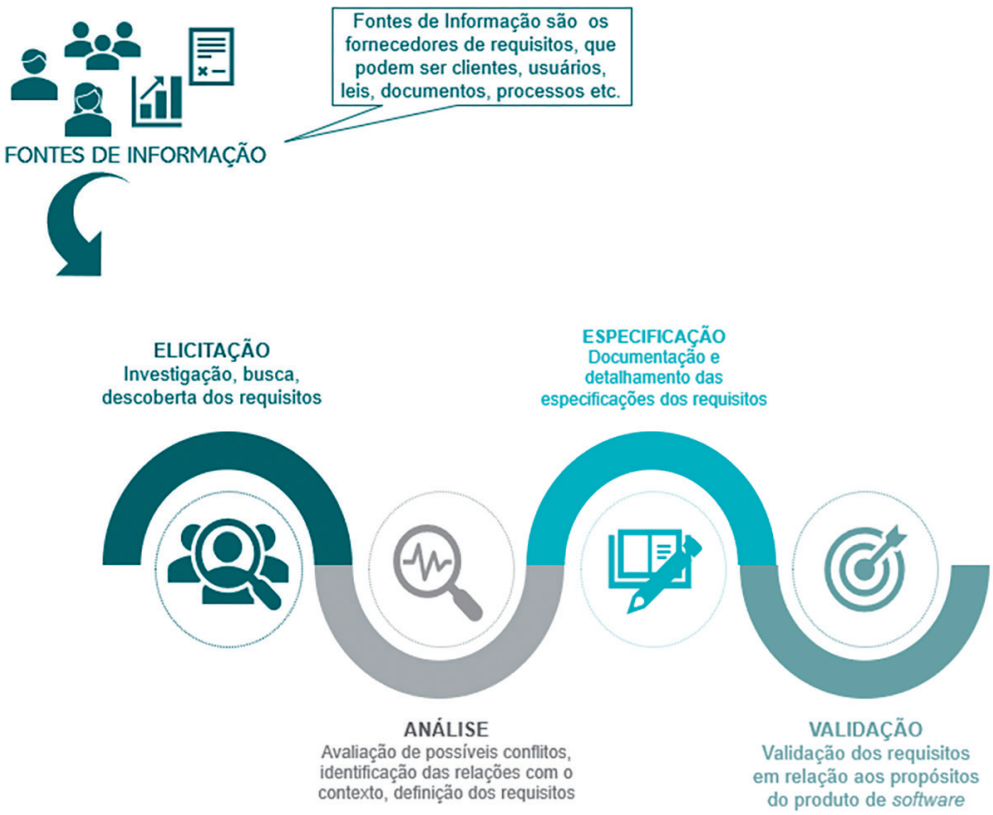


Figura 1. Atividades envolvidas no desenvolvimento de requisitos.

Como você pode observar na Figura 1, o desenvolvimento de requisitos compreende as atividades de elicitação, análise, especificação e validação. Para dar início às atividades, o primeiro passo é compreender o cenário de negócios onde o projeto está inserido e identificar quais serão as fontes de informação para a obtenção dos requisitos. Fontes de informação são os fornecedores de requisitos.

Este capítulo será desenvolvido com base em um exemplo prático, porém fictício, por meio do qual serão apresentados os tópicos mais relevantes do desenvolvimento de requisitos. Veremos a seguir a descrição do cenário de negócios desse exemplo.

Cenário de negócios

Trataremos do desenvolvimento de um aplicativo para apoiar as atividades do Detran (Departamento de Trânsito), que interligará os diversos atores envolvidos nesse ecossistema, conforme ilustra a Figura 2. Os atores são os seguintes: o próprio Detran, o motorista, o proprietário, o agente de trânsito, os bancos e as clínicas médicas. Entende-se por "proprietário" o dono de um veículo automotor, que pode, em determinado momento, ser também o motorista do veículo.



Fique atento

Embora estejamos nos inspirando em um sistema real, não temos aqui compromisso com a fidelidade aos sistemas que os diversos Detrans possuem. Trata-se de um exemplo fictício para fins didáticos.

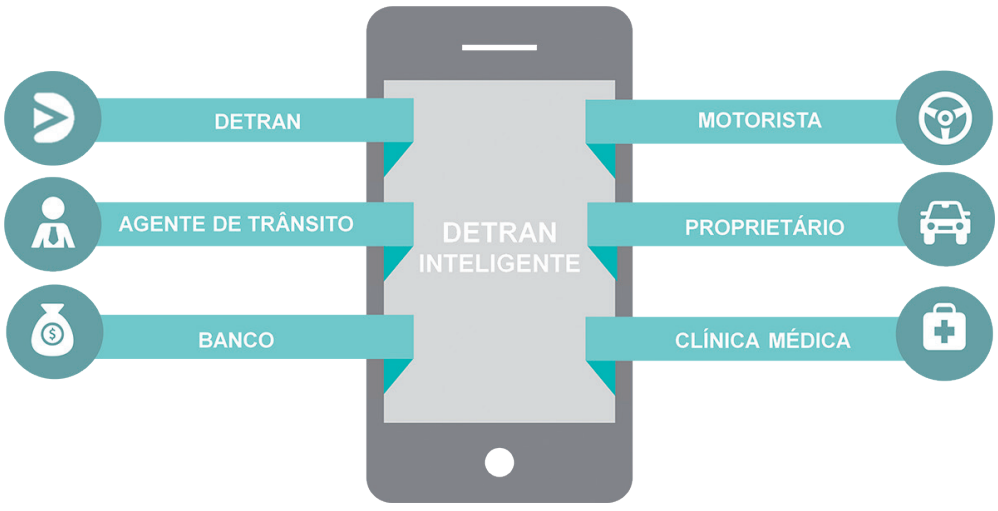


Figura 2. Contexto do aplicativo Detran Inteligente.

Suponha que você é o analista de requisitos que foi alocado para realizar as atividades relacionadas ao desenvolvimento de requisitos desse aplicativo. Você foi chamado pelo seu gerente e, a partir desse momento, deve iniciar seu trabalho seguindo as boas práticas da engenharia de requisitos. Seu objetivo é ter os requisitos especificados em um nível de detalhe suficiente para poderem ser repassados para a implementação, que contará com uma equipe experiente de desenvolvedores e testadores.

Comece recordando as dicas de Wiegers e Beatty (2013) sobre os direitos e deveres do cliente, listados no Quadro 1.

Quadro 1. Direitos e deveres do cliente

Diretos do cliente (você tem o direito a...)	**Deveres do cliente (você tem responsabilidade por...)**
1. Esperar que o analista de requisitos fale a sua língua.	1. Educar os analistas de requisitos e desenvolvedores sobre o seu negócio.
2. Esperar que o analista de requisitos aprenda sobre seu negócio e seus objetivos.	2. Dedicar o tempo que for necessário para fornecer e esclarecer requisitos.
3. Esperar que o analista de requisitos registre os requisitos de forma apropriada.	3. Ser específico e preciso ao fornecer informações sobre os requisitos.
4. Receber explicações sobre as práticas de requisitos e os produtos a serem entregues.	4. Tomar decisões oportunas sobre os requisitos quando solicitado.
5. Alterar seus requisitos.	5. Respeitar a avaliação do desenvolvedor quanto ao custo e à viabilidade dos requisitos.
6. Esperar um ambiente de respeito mútuo.	6. Estabelecer prioridades realistas de requisitos em colaboração com os desenvolvedores.
7. Ouvir ideias e alternativas para suas necessidades e para a solução delas.	7. Rever requisitos e avaliar protótipos.
8. Esperar que o analista de requisitos descreva as características que facilitarão o uso do produto.	8. Estabelecer critérios de aceitação.

(Continua)

(Continuação)

Quadro 1. Direitos e deveres do cliente

Diretos do cliente (você tem o direito a...)	Deveres do cliente (você tem responsabilidade por...)
9. Esperar que o analisa de requisitos conheça formas de ajustar os requisitos para acelerar o desenvolvimento por meio da reutilização.	9. Comunicar prontamente as alterações aos requisitos.
10. Receber um sistema que atenda às suas necessidades funcionais e expectativas de qualidade.	10. Respeitar o processo de desenvolvimento de requisitos.

Fonte: Adaptado de Wiegers e Betty (2013, tradução nossa).

Como você pode observar, o cliente tem o direito a ter um analista de requisitos presente, competente e transparente em sua comunicação. Espera ainda que haja um clima de respeito e colaboração. Por sua vez, é esperado que o cliente também cumpra com as suas obrigações como cliente e que dedique tempo suficiente para prestar as informações necessárias ao analista de requisitos, que tome as decisões no momento adequado e que, principalmente, comunique uma alteração de requisitos assim que ela for detectada.

A principal característica para que o relacionamento entre a equipe de desenvolvimento (neste momento representada pelo analista de requisitos) e o cliente seja proveitoso é a **comunicação**. Os métodos ágeis valorizam muito o papel da comunicação fluida entre as partes, em vez de disputas em torno de contratos. No caso dos métodos ágeis, quem representa os interesses do cliente é o *product owner* (PO).

Início dos trabalhos

A primeira atividade como analista de requisitos deverá ser compreender o **contexto do problema** que será resolvido pelo novo aplicativo. Uma primeira fonte dessas informações poderá ser o próprio gerente que lhe passou a atividade: pergunte tudo o que ele sabe sobre o assunto. Neste momento é possível que apareçam informações mais gerais sobre o negócio, como, por exemplo, as motivações para dar início ao desenvolvimento, possíveis resistências de pessoas envolvidas, questões de ordem política e expectativas da empresa desenvolvedora — no caso, a empresa em que você trabalha. Recolha e organize todas as informações para explorá-las depois, junto aos usuários e clientes.

Outra fonte de informação, ainda informal, pode ser algum colega que já tenha trabalhado com sistemas similares, ou mesmo a equipe que é responsável pelo sistema atualmente. Essas são pessoas que podem ajudar a mostrar o caminho inicial.

A partir desse momento, é importante **pesquisar** para conhecer um pouco mais sobre o negócio antes de dar início ao contato com os *stakeholders*. É imprescindível compreender as terminologias usadas no negócio e entender as principais regulamentações do setor, quando houver, de modo que, desde as primeiras conversas, já seja possível aprofundar as especificações com o cliente.

Quando um analista de requisitos chega bem preparado para a primeira reunião, demonstrando que "fez a tarefa de casa", ele já passa para o cliente uma boa impressão e a confiança de que eles têm um parceiro de trabalho. Invista tempo se preparando. Essa tarefa simples ajuda também a vencer resistências que possam, eventualmente, haver por parte de alguns dos clientes.

Ao pesquisar sobre o contexto é possível que você encontre o seguinte:

- **Informações sobre o negócio a que se refere o sistema:** no caso do nosso exemplo, você pode procurar pelos *websites* dos diversos Detrans do país para compreender o funcionamento deles.
- **Dados sobre sistemas similares disponíveis:** no caso do nosso exemplo, você pode procurar por sistemas para apoiar os Detrans e aplicativos para apoiar motoristas e proprietários de veículos automotores.
- **Leis e regulamentações do setor:** no nosso caso, você pode pesquisar sobre as leis de trânsito, sobre as regras para tirar a carteira de habilitação, regras para emplacamento e licenciamento de veículos, entre outras.
- **Inovações no setor:** pesquisar se existem inovações no setor que estejam sendo implementadas no país ou no exterior e que podem contribuir para as especificações do sistema que está sendo desenvolvido.

A partir daí, um primeiro contato com o cliente deverá ser realizado, com vistas a obter os detalhes específicos do projeto e **identificar quais serão as fontes de informações** para os requisitos. Continuando com o nosso exemplo, suponha que foram identificados os seguintes *stakeholders*:

- coordenador da equipe de infrações;
- coordenador da equipe de emissão de carteiras de habilitação;
- coordenador da equipe de relacionamentos institucionais (lembre-se que o sistema terá interface com clínicas médicas e com os bancos).



Fique atento

Uma das causas de falhas em projetos é não identificar todos os *stakeholders* relevantes logo no início. *Stakeholders* faltantes podem ser um grande problema para o sistema, pois seus requisitos não serão considerados e, em algum momento, quando isso aparecer, haverá retrabalho e muita insatisfação. No início do projeto é preferível ter pessoas demais envolvidas do que pessoas de menos.

Pohl e Rupp (2015) definem que as fontes de informação podem ser de três tipos: pessoas, documentos e sistemas. Preste atenção em cada uma delas. Questione a si mesmo se não deixou alguma fonte importante de fora.

2 Seleção da técnica de elicitação de requisitos

O próximo passo é a **seleção das técnicas de elicitação** de requisitos que melhor se adequam ao contexto do projeto e planejar como será a sua aplicação. Técnicas podem ser aplicadas de forma isolada ou podem ser combinadas entre si para um melhor resultado. A elicitação de requisitos está ilustrada na Figura 3.



Figura 3. Atividades da elicitação de requisitos.

A seleção das técnicas é um passo crucial para obtermos o melhor proveito de cada uma e conseguirmos captar os requisitos com menor custo e com maior qualidade. Ao selecionar as técnicas, leve em consideração o seguinte:

- a disponibilidade das fontes de informação;
- o tempo disponível para a atividade de elicitação;
- o tipo de projeto que está sendo desenvolvido;
- o tipo de informação que precisa ser capturada.

Vamos voltar ao nosso exemplo e olhar para o seguinte cenário: já existem sistemas que rodam em *mainframe* no Detran que você está atendendo. Os sistemas são desenvolvidos em Cobol, com banco de dados DB2. O aplicativo que está sendo desenvolvido precisará se comunicar com esses sistemas para obter as informações que você precisa exibir. Haverá retorno do seu aplicativo para alguns desses sistemas, como, por exemplo, para a marcação dos exames nas clínicas médicas e para o processamento de um pagamento na rede bancária.

Os coordenadores de negócios que foram mencionados na seção anterior estão disponíveis em horário comercial para interações sobre o projeto. Existe alguma possibilidade de haver conflito de prioridades, pois todos entendem que os seus requisitos são mais importantes. Tem que ser previsto o tratamento para esse risco.

Há a possibilidade deste aplicativo ser utilizado por outros Detrans do país e, portanto, foi apontado que é necessário conversar com representantes dos outros estados para entender se as necessidades coincidem e se existe a possibilidade de haver parcerias para o desenvolvimento.

Já foi feita uma tentativa anterior de desenvolver um aplicativo, mas ela não foi bem-sucedida porque os usuários achavam que o sistema era difícil de usar e que não tinha as funcionalidades que eles mais esperavam. Os parceiros (clínicas médicas e bancos) igualmente encontraram diversos problemas com o sistema anterior. Esse sistema anterior já está desativado, mas pode ser consultado. Os *stakeholders* desejam que o novo sistema seja inovador e tire a má impressão deixada pelo seu antecessor.

Para selecionar as técnicas mais apropriadas para a elicitação de requisitos, temos que olhar atentamente para as informações que foram prestadas, além das características de cada técnica. Vamos recordar algumas dicas importantes sobre quando aplicar cada técnica de elicitação de requisitos, analisando o Quadro 2.

Quadro 2. Seleção das técnicas de elicitação de requisitos

Técnica	Quando usar
Entrevista	■ Quando existem pessoas que têm o conhecimento necessário sobre o negócio e que estão disponíveis para serem entrevistadas. ■ Para captar as informações subjetivas (desabafos, sentimentos, pontos de vista) que podem ser úteis para a definição de requisitos do sistema. ■ Para identificar o fluxo de trabalho e de documentos.
Reunião	■ Para obter uma resposta rápida de várias pessoas sobre determinado assunto. ■ Quando existem problemas ou questões a serem esclarecidas, compartilhadas e expostas com os demais participantes. ■ Para envolver o grupo da tomada de decisão. ■ Para resolver situações de conflito (busca de consenso). ■ Para levantamento de alternativas e soluções para determinado problema. ■ Para analisar, avaliar e resolver problemas que envolvam pessoas de áreas ou empresas distintas. ■ Por exigência legal da empresa ou do governo.

(Continua)

(Continuação)

Quadro 2. Seleção das técnicas de elicitação de requisitos

Técnica	Quando usar
Brainstorming	■ Quando o ambiente da empresa é informal e propício para o desenvolvimento de atividades criativas. ■ Quando se deseja soluções inovadoras e criativas. ■ Quando se quer lançar novos produtos no mercado.
Observação	■ Quando o fluxo de papéis e de trabalho é relevante. ■ Quando problemas de desempenho estão relacionados à forma de executar o trabalho. ■ Quando a influência do ambiente real é importante.
Questionário	■ Se não houver tempo suficiente para entrevistar todas as pessoas relevantes. ■ Se as informações puderem ser adequadamente trabalhadas para fins estatísticos. ■ Se as pessoas estão situadas em pontos geográficos muito distantes. ■ Se, para prestar informações, houver necessidade de consultar arquivos físicos.
JAD (*joint application design*)	■ Quando for necessário reunir diversas pessoas e o tempo for curto. ■ Quando houver a possibilidade de conflitos. ■ Quando o ambiente exigir certo grau de formalismo. ■ Quando a intervenção direta do patrocinador for necessária.

Considerando as informações do cenário, uma das alternativas seria fazer a seguinte combinação de técnicas:

- Realizar inicialmente um *brainstorming* para gerar ideias inovadoras para o produto. Isso pode ser feito reunindo as pessoas já indicadas e trazendo pessoas de fora que sejam conhecidas por terem ideias disruptivas. Aqui pode também ser envolvida a equipe técnica do projeto, como os desenvolvedores.
- Realizar entrevistas com os representantes de cada classe de usuários para identificar os diversos pontos de vista (coordenador da equipe de infrações, coordenador da equipe de emissão de carteiras de habilitação e coordenador da equipe de relacionamentos institucionais). Compilar as informações para levar ao JAD.

- Analisar o sistema atual para identificar e mapear os problemas. Essa análise pode ser feita antes da conversa com os coordenadores, como preparação para ela. Pode também ser feita depois da conversa com os coordenadores, como forma de confirmar o que foi dito.
- Conduzir uma reunião no formato JAD. Dado que existe a possibilidade de haver conflitos acerca dos requisitos e prioridades, uma sessão JAD pode ajudar, uma vez que ela é mediada por um condutor neutro.
- Enviar um questionário para os outros Detrans, para identificar as similaridades e analisar a possibilidade de o aplicativo atender a mais do que um Detran.
- Realizar entrevistas complementares e observação. Pode ser necessário realizar entrevistas diretamente com representantes das clínicas associadas, por exemplo. Pode ainda ser necessário fazer uma observação para analisar os processos de negócio dentro do Detran. Isso vai depender dos resultados das técnicas anteriores.



Saiba mais

JAD ou *joint application design* é uma técnica que foi criada na IBM e que é utilizada para agilizar a obtenção e o consenso acerca dos requisitos. Ela também é chamada de *workshop* formal de requisitos.

Dependendo da organização, pode ser realizado o mapeamento do **canvas de proposta de valor**. Essa não é uma atividade obrigatória, mas é bastante recomendada como forma de fixar o olhar na entrega de valor para o cliente. Considere a possibilidade de realizar um *workshop* para mapear o canvas da proposta de valor sempre que não estiver claro para você ou para a equipe quais são os motivos pelos quais o aplicativo está sendo construído. Essa ferramenta ajuda no desenvolvimento de uma visão comum.

Aplicação das técnicas de elicitação de requisitos

Todas as técnicas exigem um tempo de **preparação** para que possam ser aplicadas com eficiência. A preparação, para qualquer uma das técnicas, envolve planejar as atividades a serem realizadas, realizar o convite ou agendamento com antecedência, confirmar o agendamento quando a data estiver mais próxima. Também preparar materiais de apoio que sejam necessários para a aplicação das técnicas de elicitação de requisitos.

3 Especificação dos requisitos de *software*

O resultado da condução da elicitação de requisitos é o conjunto de requisitos identificados. Embora existam diversas classificações para os requisitos, e eles possam ser enxergados sob diferentes níveis de abstração, aqui nós vamos focar nos requisitos funcionais e não funcionais.

Voltando ao nosso exemplo, uma lista de requisitos seria elaborada a partir da aplicação das técnicas de elicitação de requisitos que foram selecionadas. Uma lista típica de requisitos funcionais para o sistema que está sendo estudado seria a que se encontra no Quadro 3. Note que ela não é exaustiva, ou seja, ainda há muito mais requisitos a serem explorados, mas ela já dá uma boa visão do que cada classe de usuário poderá fazer no sistema. Já o Quadro 4 traz uma lista do que poderiam ser os requisitos não funcionais.

Quadro 3. Requisitos funcionais do aplicativo para o Detran

Identificador	Requisito
RF-001	O sistema deve ter um cadastro de usuário
RF-002	Todos os usuários devem se logar para operar qualquer funcionalidade no aplicativo
RF-003	O proprietário poderá cadastrar um ou mais veículos
RF-004	O proprietário poderá indicar outro condutor, no caso de uma multa
RF-005	O motorista poderá visualizar a sua CNH
RF-006	O motorista poderá consultar seus pontos na CNH

(Continua)

(Continuação)

Quadro 3. Requisitos funcionais do aplicativo para o Detran

Identificador	Requisito
RF-007	O motorista poderá consultar suas multas pagas e pendentes
RF-008	O motorista poderá recorrer de uma multa, apresentando justificativas
RF-009	O motorista poderá dar início ao processo de 1ª habilitação
RF-010	O motorista poderá realizar o pagamento das taxas para habilitação
RF-011	O motorista poderá agendar exame nas clínicas de saúde conveniadas
RF-012	A clínica de saúde poderá consultar a agenda de exames
RF-013	A clínica poderá enviar aviso de cancelamento de consulta para o motorista
RF-014	O motorista poderá enviar aviso de cancelamento de consulta para a clínica
RF-015	O motorista poderá agendar biometria para renovação de CNH
RF-016	O motorista poderá informar a sua chegada ao Detran
RF-017	O proprietário poderá tirar extrato de multas dos veículos cadastrados em seu nome
RF-018	O proprietário poderá emitir a guia do IPVA
RF-019	O proprietário poderá pagar o IPVA
RF-020	O proprietário poderá emitir a guia de licenciamento
RF- 021	O proprietário poderá pagar a guia de licenciamento
RF-022	O Detran poderá disponibilizar agenda de biometria
RF-023	O Detran poderá cancelar agenda de biometria até 24 horas antes da coleta
RF-024	O agente de trânsito poderá aplicar uma multa ao veículo
RF-025	O agente de trânsito poderá consultar dados do veículo pela placa

(Continua)

(*Continuação*)

Quadro 3. Requisitos funcionais do aplicativo para o Detran

Identificador	Requisito
RF-026	O proprietário poderá pagar taxas e multas usando cartão de débito
RF-027	O proprietário poderá pagar taxas e multas usando cartão de crédito
RF-028	O proprietário poderá pagar taxas e multas usando boleto
RF-029	Os bancos conveniados devem passar diariamente um arquivo com os retornos de cobrança
RF-030	O sistema deverá bloquear automaticamente a CNH de motoristas que excedam os pontos
(…)	(…)

Quadro 4. Requisitos não funcionais do aplicativo Detran Inteligente

Identificador	Requisito
RNF-001	O sistema deverá estar protegido por *login* e senha
RNF-002	O sistema deverá permitir que um usuário sem treinamento consiga operar qualquer função em menos de 3 minutos
RNF-003	Todas as funções devem ser acessadas diretamente com apenas um toque, por um ícone na tela do perfil de usuário
RNF-004	O tempo de resposta de qualquer transação não pode exceder 10 milissegundos
RNF-005	O tempo total que o sistema pode ficar fora do ar é de 10 minutos por mês
(…)	(…)

Caso o sistema estivesse sendo desenvolvido em um ambiente que usa métodos ágeis e histórias de usuário, os cartões das histórias de usuário seriam parecidos com o que está representado na Figura 4. Histórias de usuário são usadas para apresentar a visão dos *stakeholders* sobre as suas necessidades (PATTON, 2014).

Perceba que o cartão da história de usuário segue o padrão de escrita composto por uma frase com os seguintes elementos: (quem) + (o que) + (porque). **Quem** denota o usuário que irá utilizar a funcionalidade. **O que** denota a funcionalidade em si. **Porque** denota o motivo pelo qual aquela história de usuário é importante.

Cartões normalmente são feitos em notas adesivas de Post-it® físicas ou usando ferramentas visuais de gestão ágil automatizadas.

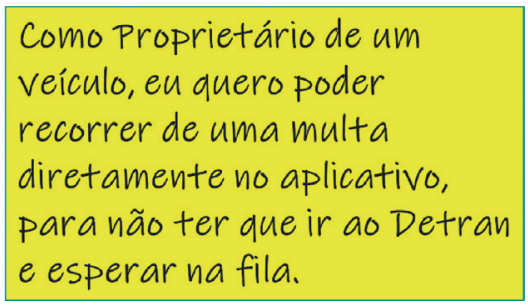


Figura 4. Exemplo de história de usuário do aplicativo do Detran.

Análise e especificação dos requisitos

Quando os requisitos estão elicitados, é hora de realizar as atividades de análise e especificação. Na análise podemos usar o diagrama de caso de uso proveniente da UML. Esse diagrama apoia a representação dos requisitos funcionais de *software*, relacionando as funcionalidades entre si e com os atores (que serão os usuários do sistema).

O diagrama é complementado pelas especificações de caso de uso, que são a forma pela qual os elementos de fluxo do caso de uso são detalhados para apoiar a implementação. Para os requisitos não funcionais são utilizadas especificações complementares que ajudam a explicitar os detalhes necessários para que esse tipo de requisito seja validado posteriormente.

Vamos retomar o nosso exemplo do aplicativo do Detran e acompanhar o diagrama de casos de uso da Figura 5. Neste exemplo demos foco aos casos de uso do Motorista e do Proprietário. O diagrama está representando apenas uma parte dos requisitos, para podermos explicar os elementos envolvidos.

O diagrama de caso de uso da Figura 5 representa os dois atores, Motorista e Proprietário; eles são representados pelo boneco palito. Existe uma relação de generalização entre eles que explicita que tudo o que o ator Motorista faz, o ator Proprietário pode fazer. Adicionalmente, o ator Proprietário pode realizar as operações sobre o veículo que ele possui. Esse tipo de relacionamento é representado por uma linha com uma seta vazada na ponta. Ela parte do elemento que herda, ou seja, do Proprietário, e vai até o Motorista. Neste caso podemos ler assim: "o Proprietário é um tipo de Motorista".

Existem no diagrama dois tipos de relacionamentos entre os casos de uso. Um deles é o que liga o caso de uso Realizar *login* a todos os demais casos de uso por meio de um <*include*>. Essa relação significa que, cada vez que o caso de uso principal for executado, também será executado o caso de uso chamado. Por exemplo, se o ator Motorista quiser consultar uma multa, ele deverá estar logado no sistema. Essa é uma regra que estava estabelecida na lista de requisitos do Quadro 3, como RF-002 — Todos os usuários devem se logar para operar qualquer funcionalidade no aplicativo.

O outro tipo de relacionamento é o <*extend*>, que denota que uma relação é opcional, que ela pode ou não acontecer. Vamos acompanhar no diagrama o caso de uso Pagar multas. Quando o Motorista executar o caso de uso Consultar multas, ele poderá pagar a multa, ou não. Isso é demonstrado pela relação de <*extend*> entre eles.



Fique atento

O sentido das setas pontilhadas dos relacionamentos de <*include*> e <*extend*> é específico. Enquanto no <*include*> a seta aponta do caso de uso que inclui (chamado de caso de uso base) para o caso de uso que é incluído, no <*extend*> é o contrário, ou seja, a seta aponta do caso de uso que estende para o caso de uso que é estendido (caso de uso base). Isso nos ajuda a ler e compreender a semântica do diagrama.

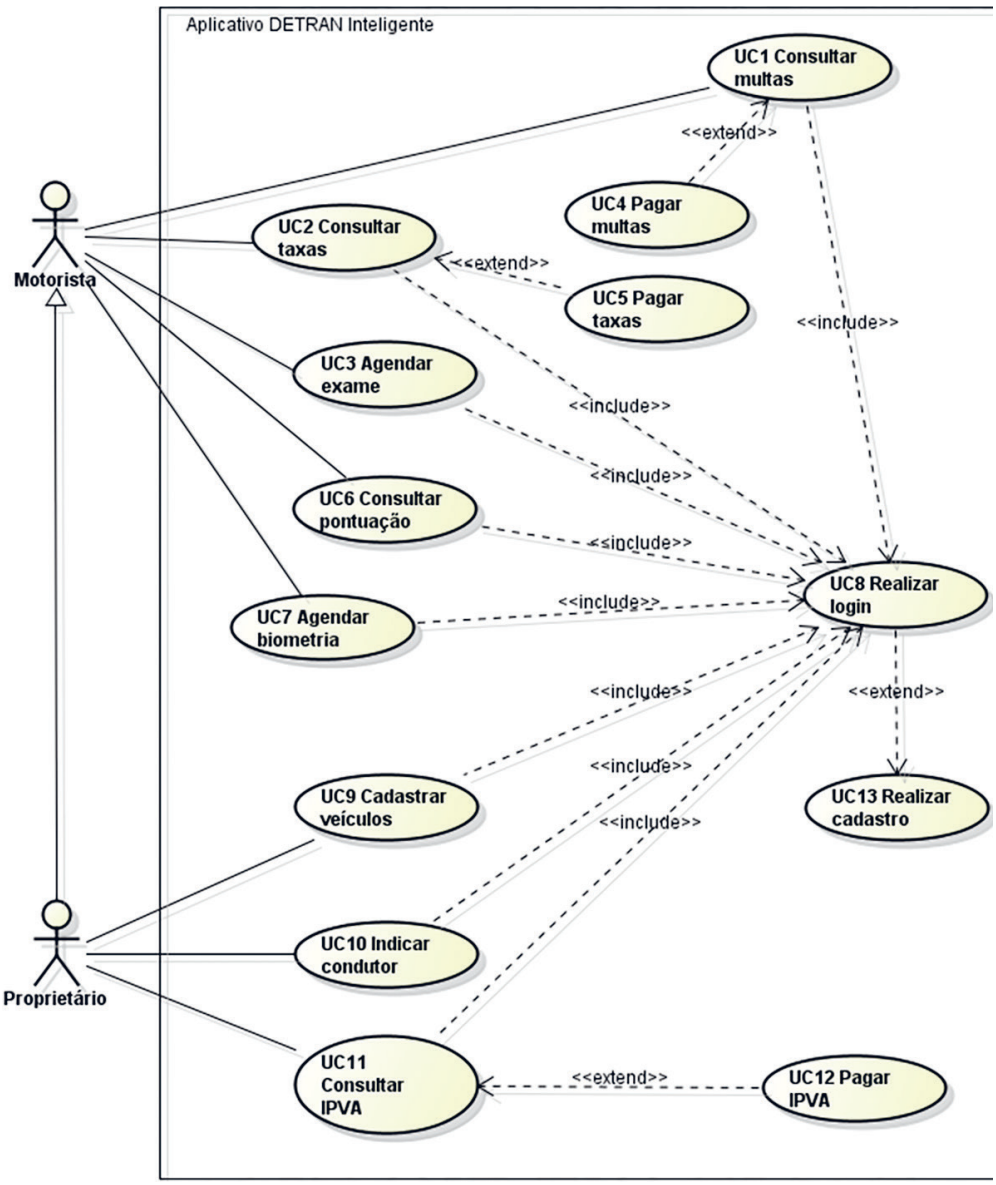


Figura 5. Diagrama de caso de uso do Sistema Detran Inteligente.

Aqui é importante ressaltar o conceito de caso de uso. Um **caso de uso** é uma execução completa de uma funcionalidade que entrega um valor para um ator. Por esse motivo não existe uma relação de um para um em relação aos requisitos funcionais. É possível que determinado requisito se desdobre em vários casos de uso, especialmente quando se tratar de um requisito de mais alto nível. Também é possível que um caso de uso esteja relacionado a vários requisitos mais detalhados.

Para complementar esse diagrama, teríamos que representar os demais atores envolvidos e os casos de uso que expressam todas as funcionalidades que estão listadas no Quadro 3.

A pergunta que surge é a seguinte: será que a lista de requisitos e o diagrama de casos de uso são suficientes para que o desenvolvedor possa programar o sistema? A resposta é depende, mas é pouco provável. Muitas vezes a equipe é muito experiente e já conhece bem o domínio de negócios. Nesses casos, menos especificações são necessárias. No entanto, no caso de um sistema deste porte, provavelmente não seja suficiente, pois detalhes das regras de negócio não aparecem no diagrama.

Para complementar o entendimento, as especificações de caso de uso precisam ser realizadas de modo que os detalhes que são necessários ao implementador sejam especificados. Aqui deve haver um balanceamento entre o que precisa estar detalhado e o que não é necessário. Um artefato só faz sentido para um sistema se ele for utilizado por alguém. Se não, ele não deve ser construído.

Vamos ver um exemplo do detalhamento de um dos casos de uso do sistema do Detran, representado no Quadro 5. Como você pode observar, a especificação é representada por três tipos de fluxo: o fluxo principal, o fluxo alternativo e o fluxo de exceção.

Um **fluxo básico** ou **fluxo principal** especifica o diálogo entre o ator e o sistema em uma situação de realização da funcionalidade completa e na qual tudo funciona conforme o esperado. Por esse motivo, o fluxo principal é também chamado de caminho feliz. Note que são representados os diálogos que acontecem entre o ator e o sistema, como se fosse realmente uma conversa. Os **fluxos alternativos**, como o próprio nome sugere, são as possibilidades de cenários alternativos ao fluxo principal, conforme escolhas do usuário ou situações específicas de regras de negócio do sistema. Não são os fluxos para o tratamento de erro, mas sim escolhas que podem ser feitas pelo usuário. Já os **fluxos de exceção** demonstram como o sistema reagirá na presença de situações inesperadas, com erros, por exemplo. Os fluxos de exceção podem ocorrer dentro do fluxo principal ou dentro de um fluxo alternativo.

Vamos retomar o nosso caso do aplicativo para o Detran, especificando um dos casos de uso, Consultar multas. Um modelo que pode ser utilizado para a especificação é o que está representado no Quadro 6.

Quadro 6. Especificação de caso de uso Consultar multas

Identificador único do caso de uso	Consultar multas
Descrição	Permite ao Motorista e ao Proprietário consultar multas
Ator principal	Motorista e Proprietário
Atores secundários	×
Pré-condições	O caso de uso Realizar *login* ter sido executado com sucesso e o comprador estar logado
Gatilho	Comprador Logado seleciona Consultar multas
Fluxo principal	
Ações do ator	**Ações do sistema**
	1 – Sistema apresenta as multas do Motorista
2 – Motorista seleciona a multa que deseja consultar	
	3 – Sistema exibe os detalhes da multa selecionada
4 – Motorista manifesta o desejo de pagar a multa	
	5 – Sistema desvia para o fluxo alternativo A1
	6 – Sistema altera a situação da multa para paga
	7 – Sistema retorna para a exibição das multas do Motorista
8 – Motorista solicita encerrar a consulta	
	9 – Sistema retorna para o menu principal

(Continua)

(Continuação)

Quadro 6. Especificação de caso de uso Consultar multas.

Fluxo alternativo A1	
	A1.1 – Sistema chama o caso de uso Pagar multas (A1) A1.2 – Sistema recebe o retorno do caso de uso Pagar multas (E1) A1.3 – Sistema retorna para o fluxo principal
Fluxo de exceção E1	
	E1.1 – Caso de uso Pagar multas retornou com erro e o sistema exibe mensagem: "Não foi possível completar o pagamento da multa. Favor tentar mais tarde."
Pós-condição	Não se aplica

Note que, nesse exemplo, o fluxo principal é o caso no qual o Motorista consulta as suas multas e pode ou não optar por pagar a multa. Se ele optar por pagar a multa, o sistema aciona o fluxo alternativo (A1), que chama o caso de uso Pagar multas. Veja que aqui temos representado um ponto de extensão, que no diagrama de caso de uso é desenhado como um relacionamento de <*extend*>. Caso ocorra algum erro, esse erro é tratado pelo fluxo de exceção E1. As nomenclaturas para essas chamadas aos demais fluxos podem variar de acordo com a empresa ou com o padrão que ela estabelecer. O importante é manter íntegro o conceito de cada um dos elementos.

Faz parte da análise também a preocupação com a **especificação dos requisitos não funcionais**, que são chamados de requisitos de qualidade. Esses requisitos normalmente são transversais a todo o sistema e não são específicos para uma funcionalidade. Para apoiar essa especificação, a família de normas internacionais ISO/IEC 25000 (ISO/IEC, 2014) pode ser utilizada. Lá você encontrará diversos indicadores e exemplos de aplicação. Geralmente esses requisitos se referem a questões percebidas pelo usuário quando está usando o sistema, a chamada qualidade em uso, e também por outros atributos não tão visíveis para ele, como a qualidade interna do produto.

No nosso sistema exemplo, temos os requisitos não funcionais listados no Quadro 4. Um desses requisitos trata da facilidade de uso que o aplicativo deve ter, uma vez que, de acordo com o RNF-002, o sistema deverá permitir que um usuário sem treinamento consiga operar qualquer função em menos de 3 minutos. Esse é um requisito considerado de usabilidade, ou seja, o usuário precisa compreender o funcionamento do aplicativo sem receber treinamento e realizar as funções sem ajuda. Poderíamos detalhar esse requisito não funcional por meio dos atributos que permitirão que ele seja testado de forma mais assertiva, conforme aparece no Quadro 7. Nesse caso específico, foi estabelecido que, ao submeter o aplicativo para um conjunto de usuários típicos, 90% deles deve ser capaz de realizar as funções descritas.

Quadro 7. Especificação de requisito não funcional

Identificador	Detalhamento
RNF-002	O sistema deverá permitir que um usuário sem treinamento consiga operar qualquer função em menos de 30 segundos. ■ Um usuário típico do sistema deverá ser capaz de compreender o funcionamento do sistema sem necessitar de treinamento. ■ Ao realizar o teste com 10 usuários típicos, 90% tem que ser capaz de consultar as multas sem a necessidade de acionar o botão de Ajuda. ■ Ao realizar o teste com 10 usuários típicos, 90% deve ser capaz de finalizar o pagamento de uma multa sem a necessidade de acionar o botão de Ajuda. (…)

A forma como podemos definir o que é um usuário típico é por meio do mapeamento de personas. Esse é um conceito que vem da área de *marketing*, mas que vem sendo utilizado no desenvolvimento de *software*, especialmente em *startups*. Trata-se de criar um personagem com todas as características de um usuário típico do sistema. Isso apoiará a equipe técnica na definição de detalhes referentes ao uso do produto, como a interface, por exemplo. Se a descrição da persona apontar para alguém que tem grande intimidade com a tecnologia, a interface será projetada de uma forma. Se apontar para um grupo de usuários com pouca intimidade com a tecnologia, os cuidados com a interface terão que ser maiores.



Referências

ISO/IEC. *ISO/IEC 25000:2014(E):* systems and software engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE): guide to SQuaRE. Switezland: ISO, 2014.

PATTON, J. *User story mapping*: discover the whole story, build the right product. Sebastopol: O´Reilly, 2014.

POHL, K.; RUPP, C. *Requirements engineering fundamentals:* a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leituras recomendadas

DENNIS, A.; WIXON, B.; ROTH, R. *System analysis and design*. 5. ed. Hoboken: John Wiley and Sons, 2012.

PRESSMANN, R.; MAXIM, B. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

Parte 4

Gerenciamento de Requisitos em um Cenário de Negócios

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Gerenciamento de requisitos em um cenário de negócios

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Planejar a verificação e validação de requisitos em um cenário de negócios.
- Construir a matriz de rastreabilidade bidirecional de requisitos.
- Analisar os impactos de uma solicitação de mudanças em um cenário de negócios.

Introdução

Uma das mudanças mais expressivas de hábitos ocorrida como consequência do avanço da tecnologia foi a utilização massiva de *smartphones*. Para a maioria das pessoas, ficar sem o *smartphone*, mesmo que por algumas horas, é sinônimo de pânico. Criou-se uma dependência de suas funcionalidades que, muitas vezes, é considerada até nociva. Mas a realidade é que o uso do aparelho para as atividades cotidianas já é irreversível, sejam elas pessoais, sejam profissionais.

Na esteira dessa utilização, a maioria das empresas que presta serviços passou a oferecer algumas de suas opções por meio do celular. Esse é o caso, por exemplo, das companhias aéreas, que, além da compra de passagens, oferecem todas as facilidades para a realização do *check-in* e, até mesmo, para a antecipação de voos. Não é mais necessário imprimir o cartão de embarque, basta apresentar a imagem no celular para ser escaneada no momento do embarque. Isso sem falar nas empresas que cresceram por conta dos dispositivos móveis, como as que ofertam os serviços por assinatura de transmissão de músicas e vídeo por *streaming*, como Spotify e Netflix.

Não tardou para que a administração pública passasse a reconhecer a importância de oferecer os seus serviços ao cidadão por meio do celular. Atualmente, já é possível resolver uma série de trâmites junto aos órgãos do governo diretamente pelo celular e sem que seja necessária a presença física em uma repartição, quer na esfera municipal, quer na estadual ou federal. Documentos pessoais também passaram a ser oferecidos de forma digital. Este capítulo vai focar no desenvolvimento de um aplicativo para apoiar o Detran (Departamento de Trânsito) na comunicação com seus usuários, os proprietários e condutores de veículos automotores. O objetivo é percorrer as atividades da engenharia de requisitos, uma das partes fundamentais para a qualidade dos produtos de *software*, apresentando os conceitos associados de forma prática.

A engenharia de requisitos pode ser entendida como um conjunto de práticas que, associadas, visam à garantia da qualidade do produto de *software*. Didaticamente, podemos considerar dois grupos de atividades: o desenvolvimento dos requisitos e o gerenciamento dos requisitos. No desenvolvimento dos requisitos estão concentradas as atividades relacionadas à compreensão do contexto do projeto, elicitação, análise, especificação e validação dos requisitos. Já no gerenciamento dos requisitos estão concentradas as atividades relacionadas com a rastreabilidade, acompanhamento do *status*, controle de versão e controle de mudanças nos requisitos.

No Capítulo Gerenciamento de requisitos em um cenário de negócios, da obra *Engenharia de requisitos*, você vai aprender sobre a validação de requisitos, uma importante etapa da engenharia de requisitos que envolve atividades de verificação e validação de requisitos. Vai aprender sobre a matriz de rastreabilidade aplicada a um cenário de negócios, além de ler sobre a análise de impacto de solicitações de mudança.

1 Planejamento da verificação e da validação

A engenharia de requisitos é uma das disciplinas mais importantes da engenharia de *software*. Sem a base fornecida por requisitos bem definidos, bem especificados e confiáveis, não é possível construir bons produtos de *software*.

A Figura 1 apresenta as atividades envolvidas no desenvolvimento dos requisitos. Embora possa, no primeiro momento, parecer que ela esteja representando atividades sequenciais, que se desenrolam no formato cascata, este não é o caso. Na verdade, trata-se de atividades de natureza iterativa e incremental, que podem ocorrer diversas vezes ao longo do desenvolvimento de um projeto.

Também não estamos aqui limitando a interpretação a uma forma tradicional de gerenciamento, ou seja, as atividades relacionadas a requisitos terão que acontecer, seja em um ambiente de gestão mais tradicional, seja em um ambiente mais ágil.



Figura 1. Atividades envolvidas no desenvolvimento de requisitos.

Como você pode observar na Figura 1, o desenvolvimento de requisitos compreende as atividades de elicitação, análise, especificação e validação. Nossa ênfase nesta sessão será sobre a validação de requisitos. De acordo com o SWEBOK (BOURQUE; FAIRLEY, 2014), as atividades de validação compreendem também as atividades de verificação, portanto aqui trataremos das duas coisas que, juntas, costumam ser chamadas de atividades de V&V. É importante, no entanto, compreender a diferença entre elas: **verificação** trata de garantir que o produto está sendo construído de forma correta, enquanto **validação** trata de garantir que o produto certo foi construído. Validar implica em ter a participação do usuário ou de um representante dele para garantir que o produto atenderá ao seu uso pretendido.



Saiba mais

O SWEBOK ressalta que "[...] os requisitos podem ser validados para assegurar que o engenheiro de *software* compreendeu os requisitos; é também importante verificar que o documento de requisitos está em conformidade com os padrões da empresa e que ele é compreensível, consistente e completo" (BOURQUE; FAIRLEY, 2014, documento *on-line*).

Este capítulo será desenvolvido com base em um exemplo prático, porém fictício, por meio do qual serão apresentados os tópicos mais relevantes das atividades de verificação e validação, além das atividades de gerenciamento que serão discutidas na próxima seção. Veremos a seguir a descrição do cenário de negócios desse exemplo.

Cenário de negócios

Abordaremos o desenvolvimento de um aplicativo para apoiar as atividades oferecidas por um órgão da administração pública, neste caso, o Detran (Departamento de Trânsito), que interligará os diversos atores envolvidos nesse ecossistema, conforme ilustra a Figura 2. São eles: o próprio Detran, o motorista, o proprietário, o agente de trânsito, os bancos e as clínicas médicas. Entende-se por proprietário o dono de um veículo automotor, que pode, em determinado momento, ser também o motorista do veículo.



Fique atento

Embora estejamos nos inspirando em um sistema real, não temos aqui compromisso com a fidelidade aos sistemas que os diversos Detrans possuem. Trata-se de um exemplo fictício para fins didáticos.



Figura 2. Contexto do aplicativo Detran Inteligente.

Suponha que você é o analista de requisitos que foi alocado para realizar as atividades relacionadas ao desenvolvimento de requisitos desse aplicativo. Você já identificou os requisitos funcionais e não funcionais, já elaborou o diagrama de casos e de uso e já descreveu as especificações dos casos de uso e dos requisitos não funcionais.

Neste momento, você está alocado para planejar e realizar as atividades relacionadas à verificação e à validação dos requisitos. Seu objetivo é garantir que os requisitos especificados estão completos, consistentes e podem ser repassados para a implementação, que contará com uma equipe experiente de desenvolvedores e testadores. Além disso, você deverá garantir o envolvimento dos *stakeholders* relevantes no processo de homologação dos requisitos.

Início dos trabalhos

Primeiramente, vamos resgatar o que já foi produzido até o momento, de modo que você possa seguir com as suas atividades. No início do projeto, foram identificados os seguintes *stakeholders*, que serviram como fontes de informação para os requisitos:

- coordenador da equipe de infrações;
- coordenador da equipe de emissão de carteiras de habilitação;
- coordenador da equipe de relacionamentos institucionais (lembre-se que o sistema terá interface com clínicas médicas e com os bancos).



Fique atento

As atividades de validação de requisitos devem envolver os *stakeholders* relevantes, para garantir que o produto que está sendo construído atenderá às suas necessidades. Uma das causas de falhas em projetos é não identificar todos os *stakeholders* relevantes logo no início ou não os envolver de forma adequada ao longo do processo de desenvolvimento.

Embora nos métodos ágeis esse papel caiba ao *product owner* (PO), é preciso garantir que ele efetivamente está representando todos os interesses dos envolvidos, caso contrário, o produto não satisfará o seu uso pretendido.

Informações adicionais também foram passadas no que se refere ao contexto onde o aplicativo está inserido. Existem sistemas que rodam em *mainframe* no Detran que você está atendendo, que foram desenvolvidos em Cobol com DB2 e precisam se comunicar com o aplicativo que está sendo desenvolvido, de modo a obter as informações que precisam ser exibidas. Haverá retorno do seu aplicativo para alguns desses sistemas, como, por exemplo, para a marcação dos exames nas clínicas médicas e para o processamento de um pagamento na rede bancária.

Os coordenadores de negócios mencionados anteriormente estão disponíveis em horário comercial para interações sobre o projeto. Há a possibilidade de esse aplicativo ser utilizado por outros Detrans do país e, portanto, foi apontado que é necessário conversar com representantes dos outros estados

para entender se as necessidades coincidem e se existe a possibilidade de haver parcerias para o desenvolvimento. Caso seja um requisito primordial, esses parceiros, mesmo fisicamente distantes, precisarão ser envolvidos nas atividades de validação.

Os *stakeholders* desejam que o novo sistema seja inovador e tire a má impressão deixada pelo seu antecessor, uma vez que, na tentativa anterior de desenvolver um aplicativo, os usuários acharam que o sistema era difícil de usar e que não tinha as funcionalidades que eles mais esperavam. Os parceiros (clínicas médicas e bancos) igualmente encontraram diversos problemas com o sistema anterior. Essa é uma informação que pode impactar sobremaneira os esforços de desenvolvimento, mas também o planejamento para a sua validação.

Em seguida foi produzida a lista dos requisitos funcionais, o diagrama de casos de uso, a lista de requisitos não funcionais e a especificação e requisitos não funcionais. Esses artefatos serão apresentados juntamente com as técnicas utilizadas para fazer a sua verificação e validação, nas próximas seções.

Verificação de requisitos

A primeira ação de planejamento para a verificação de requisitos é recordar o que é a verificação de requisitos e quais são as técnicas que podem ser utilizadas. De acordo com a norma internacional ISO/IEC/IEEE 12207 (ISO/IEC/IEEE, 2017, p. 10–11), verificação é a "confirmação, por meio do fornecimento de evidência objetiva, que os requisitos especificados foram atendidos." De acordo com o CMMI-DEV 2.0 (CMMI INSTITUTE, 2018, p. 525), a verificação "endereça que o produto de trabalho ou a solução refletem adequadamente os requisitos especificados, isto é, 'você está construindo certo'".

Existem diversas formas de fazer a verificação dos requisitos. Uma delas é a revisão por pares, que nada mais é do que um par realizando a revisão sobre o trabalho do colega, com o objetivo de identificar inconsistências ou anomalias. Elas podem ser feitas por meio de inspeções, *walkthroughs* estruturados, auditorias ou outras formas de revisão. Sua aplicação vai depender do grau de formalismo desejado para a revisão. Inspeções são consideradas revisões mais formais e têm um formato próprio para acontecer.

As revisões também podem ser feitas seguindo diferentes abordagens. Podem ser baseadas em *checklists*, em cenários de execução, com foco na perspectiva de algum usuário, com foco em uma funcionalidade específica ou ser feita até mesmo de forma *ad hoc*.

Neste exemplo vamos considerar a aplicação de um *checklist* para avaliar os artefatos que foram produzidos até o momento. São eles:

- lista de requisitos;
- diagrama de casos de uso;
- especificação de casos de uso;
- especificação de requisitos não funcionais.

O Quadro 1 apresenta a lista de requisitos que foi produzida para o aplicativo do Detran que está sendo analisado neste capítulo. Por uma questão de espaço, ela foi simplificada.

Quadro 1. Requisitos funcionais do aplicativo Detran Inteligente

Identificador	Requisito
RF-001	O sistema deve ter um cadastro de usuário
RF-002	Todos os usuários devem se logar para operar qualquer funcionalidade no aplicativo
RF-003	O proprietário poderá cadastrar um ou mais veículos
RF-004	O proprietário poderá indicar outro condutor, no caso de uma multa
RF-005	O motorista poderá visualizar a sua CNH
RF-006	O motorista poderá consultar seus pontos na CNH
RF-007	O motorista poderá consultar suas multas pagas e pendentes
RF-008	O motorista poderá recorrer de uma multa, apresentando justificativas
RF-009	O motorista poderá dar início ao processo de 1ª habilitação
RF-010	O motorista poderá realizar o pagamento das taxas para habilitação
RF-011	O motorista poderá agendar exame nas clínicas de saúde conveniadas
RF-012	A clínica de saúde poderá consultar a agenda de exames

(Continua)

(Continuação)

Quadro 1. Requisitos funcionais do aplicativo Detran Inteligente

Identificador	Requisito
RF-013	A clínica poderá enviar aviso de cancelamento de consulta para o motorista
RF-014	O motorista poderá enviar aviso de cancelamento de consulta para a clínica
RF-015	O motorista poderá agendar biometria para renovação de CNH
RF-016	O motorista poderá informar a sua chegada ao Detran
RF-017	O proprietário poderá tirar extrato de multas dos veículos cadastrados em seu nome
RF-018	O proprietário poderá emitir a guia do IPVA
RF-019	O proprietário poderá pagar o IPVA
RF-020	O proprietário poderá emitir a guia de licenciamento
RF-021	O proprietário poderá pagar a guia de licenciamento
RF-022	O DETRAN poderá disponibilizar agenda de biometria
RF-023	O DETRAN poderá cancelar agenda de biometria até 24 horas antes da coleta
RF-024	O agente de trânsito poderá aplicar uma multa ao veículo
RF-025	O agente de trânsito poderá consultar dados do veículo pela placa
RF-026	O proprietário poderá pagar taxas e multas usando cartão de débito
RF-027	O proprietário poderá pagar taxas e multas usando cartão de crédito
RF-028	O proprietário poderá pagar taxas e multas usando boleto
RF-029	Os bancos conveniados devem passar diariamente um arquivo com os retornos de cobrança
RF-030	O sistema deverá bloquear automaticamente a CNH de motoristas que excedam os pontos
(...)	(...)

Assim que a lista de requisitos é produzida, já é possível utilizar o *checklist* para realizar a verificação. Um exemplo de *checklist* se encontra no Quadro 2. Os itens desse *checklist* foram adaptados de Wiegers e Beatty (2013), complementados com itens da norma internacional ISO/IEC/IEEE 29148 (ISO/IEC/IEEE, 2018).

O exemplo aqui apresentado refere-se à verificação realizada sobre o requisito RF-003. Note que o item 2 foi apontado como "Não se aplica". Como neste momento o *checklist* está sendo usado para a revisão por pares feita com o olhar de um colega (par), não é possível afirmar que "O requisito reflete o que o usuário, cliente ou seus representantes desejam?". Essa pergunta poderá ser respondida pelos *stakeholders* relevantes, no momento da validação. O item 6, que se refere à priorização, também não foi atendido, uma vez que não há prioridade atribuída ao requisito. Nesse caso foi assinalada a coluna "Não".

Quadro 2. *Checklist* para a lista de requisitos — aplicado ao requisito RF-003

RF-003 — O proprietário poderá cadastrar um ou mais veículos				
Atributo	**Definição**	**Sim**	**Não**	**Não se aplica**
1. Completo	O requisito está especificado de forma completa e que possibilita que o desenvolvedor o implemente?	×		
2. Correto	O requisito reflete o que o usuário, cliente ou seus representantes desejam?			×
3. Único	O requisito descreve uma única capacidade, característica, restrição ou atributo de qualidade?	×		
4. Viável	O requisito é viável técnica e financeiramente de ser implementado, de acordo com as restrições do projeto?	×		

(Continua)

(Continuação)

Quadro 2. *Checklist* para a lista de requisitos — aplicado ao requisito RF-003

RF-003 — O proprietário poderá cadastrar um ou mais veículos				
Atributo	**Definição**	**Sim**	**Não**	**Não se aplica**
5. Necessário	O requisito tem um motivo de existir, que é representado pelo seu relacionamento a uma fonte de informação e a um objetivo de negócio?	×		
6. Priorizado	O requisito tem uma prioridade atribuída para que possa ser alocado a uma versão do *software*?		×	
7. Não ambíguo	O requisito não contém ambiguidades que levem os *stakeholders* a interpretá-lo de forma diferente?	×		
8. Verificável	O requisito é possível de ser verificado posteriormente quanto à sua implementação?	×		
9. Conforme	O requisito está em conformidade com os padrões de especificação estabelecidos?	×		
Observações: nenhuma				

Fonte: Adaptado de Wiegers e Beatty (2013) e ISO/IEC/IEEE (2018).

O segundo artefato que foi desenvolvido no projeto foi o diagrama de casos de uso, representado na Figura 3. O diagrama de casos de uso, embora simples, tem um grande poder de comunicação, mas ele só atenderá os seus objetivos se seguir a semântica proposta pela UML. O diagrama representado na Figura 3 foi simplificado e não representa todo o escopo do sistema, por uma questão de espaço, mas serve aos nossos propósitos didáticos.

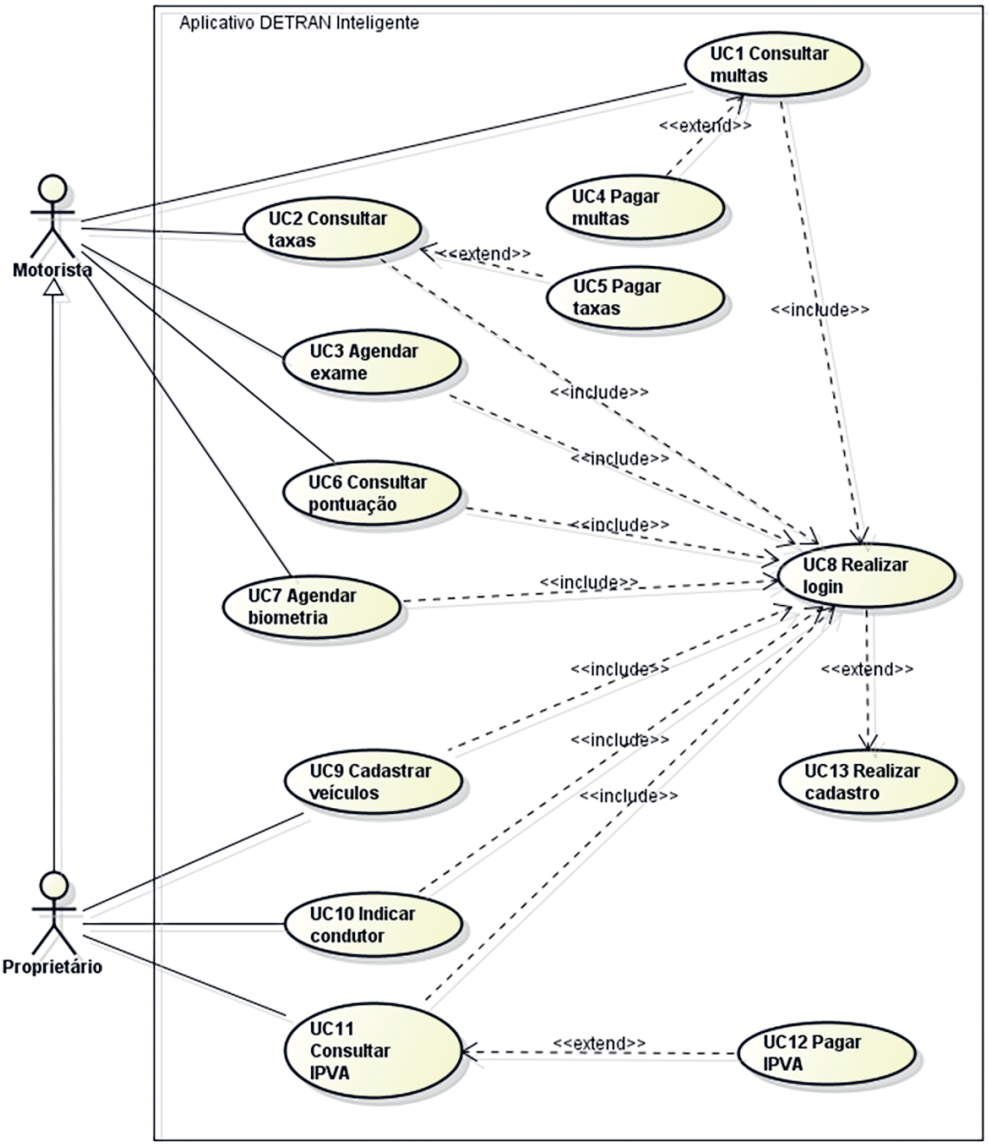


Figura 3. Diagrama de caso de uso do Aplicativo DETRAN Inteligente v1.0.

O Quadro 3 apresenta o *checklist* que foi aplicado para a verificação do diagrama de casos de uso. Como você pode observar, o item 1 do *checklist* não foi atendido, pois nem todos os atores foram representados no diagrama. O mesmo ocorre com o item 6, uma vez que nem todos os casos de uso estão representados no diagrama. Os itens 10 e 11 foram assinalados como "não se aplica", uma vez que foram identificados relacionamentos de generalização entre os casos de uso. Os demais itens foram todos atendidos.

Quadro 3. *Checklist* para o diagrama de casos de uso do sistema Detran Inteligente

Diagrama de casos de uso do sistema Detran Inteligente v1.0				
Atributo	**Definição**	**Sim**	**Não**	**Não se aplica**
1. Atores (escopo)	Todos os atores estão representados no diagrama?		×	
2. Atores (formato)	Todos os atores estão representados por um boneco palito e um texto com a identificação do ator?	×		
3. Relacionamentos de generalização entre atores (representação)	O relacionamento de generalização entre os atores é representado por uma seta vazada que aponta do ator filho para o ator pai?	×		
4. Relacionamentos de generalização entre atores (significado)	O relacionamento de generalização expressa corretamente a intenção de herança entre os atores envolvidos?	×		
5. Casos de uso (representação)	Todos os casos de uso são representados por elipses e identificados por meio de um verbo no infinitivo seguido de um complemento?	×		
6. Casos de uso (escopo)	Todos os casos de uso que representam as funcionalidades que os *stakeholders* desejam estão representados?		×	
7. Relacionamentos dos casos de uso com atores	Todos os casos de uso estão ligados a pelo menos um ator?	×		
8. Relacionamentos de <*include*> entre casos de uso	Os relacionamentos de <*include*> estão representados por setas pontilhadas que partem do caso de uso principal para o caso de uso incluído?	×		

(Continua)

(Continuação)

Quadro 3. *Checklist* para o diagrama de casos de uso do sistema Detran Inteligente

Diagrama de casos de uso do sistema Detran Inteligente v1.0				
Atributo	**Definição**	**Sim**	**Não**	**Não se aplica**
9. Relacionamentos de <*extend*> entre casos de uso	Os relacionamentos de <*extend*> estão representados por setas pontilhadas que partem do caso de uso que estende para o caso de uso principal?	×		
10. Relacionamentos de generalização entre casos de uso (representação)	O relacionamento de generalização entre os casos de uso é representado por uma seta vazada que aponta do caso de uso filho para o caso de uso pai?			×
11. Relacionamentos de generalização entre casos de uso (significado)	O relacionamento de generalização expressa corretamente a intenção de herança entre os casos de uso envolvidos?			×
Observações: o diagrama analisado ainda não estava completo e com todos os casos de uso do aplicativo.				

Como complemento ao diagrama de caso de uso, foram realizadas as especificações de caso de uso que visam a detalhar como serão realizados os cenários de execução. O Quadro 4 representa a especificação de caso de uso do UC1. Consultar multas.

Quadro 4. Especificação de caso de uso UC1. Consultar multas

Identificador único do caso de uso	Consultar multas
Descrição	Permite ao Motorista e ao Proprietário consultar multas
Ator Principal	Motorista e Proprietário
Atores Secundários	×
Pré-condições	O caso de uso UC13. Realizar *login* ter sido executado com sucesso e o comprador estar logado
Gatilho	Comprador Logado seleciona Consultar multas
Fluxo principal	
Ações do ator	**Ações do sistema**
	1 – Sistema apresenta as multas do Motorista
2 – Motorista seleciona a multa que deseja consultar	
	3 – Sistema exibe os detalhes da multa selecionada
4 – Motorista demonstra o desejo pagar a multa	
	5 – Sistema desvia para o fluxo alternativo A1
	6 – Sistema altera a situação da multa para <paga>
	7 – Sistema retorna para a exibição das multas do Motorista
8 – Motorista solicita encerrar a consulta	
	9 – Sistema retorna para o menu principal

(Continua)

(Continuação)

Quadro 4. Especificação de caso de uso UC1. Consultar multas

Fluxo alternativo A1	
	A1.1 – Sistema chama o caso de uso UC4. Pagar multas (A1) A1.2 – Sistema recebe o retorno do caso de uso UC4. Pagar multas (E1) A1.3 – Sistema retorna para o fluxo principal
Fluxo de exceção E1	
	E1.1 – Caso de uso UC4. Pagar multas retornou com erro e o sistema exibe mensagem: "Não foi possível completar o pagamento da multa. Favor tentar mais tarde."
Pós-condição	Não se aplica

O Quadro 5 apresenta o resultado da aplicação do *checklist* sobre a especificação do caso de uso UC1. Consultar multas. Como você pode observar, todos os itens foram atendidos. O item 3 e o item 10 foram assinalados como "Não se aplica", pois neste caso não havia necessidade desses elementos.

Quadro 5. *Checklist* para a especificação de casos de uso do UC1. Consultar multas

Atributo	Definição	Sim	Não	Não se aplica
1. Identificação do caso de uso	O caso de uso tem um identificador único e um texto contendo um verbo no infinitivo e um complemento?	×		
2. Ator principal	O ator principal está identificado corretamente?	×		

(Continua)

(Continuação)

Quadro 5. *Checklist* para a especificação de casos de uso do UC1. Consultar multas

Atributo	Definição	Sim	Não	Não se aplica
3. Ator secundário	O ator secundário (se houver) está identificado corretamente e participa do caso de uso juntamente com o ator principal?			×
4. Pré-condição	As pré-condições (se houver) estão descritas corretamente?	×		
5. Fluxo principal	Todos os passos do fluxo principal estão numerados sequencialmente e descrevem claramente o diálogo entre o ator e o sistema?	×		
6. Fluxos alternativos (descrição)	Todos os fluxos alternativos (se houver) estão numerados sequencialmente e descrevem claramente o diálogo entre o ator e o sistema?	×		
7. Fluxos alternativos (chamadas)	Para todos os fluxos alternativos (se houver) estão identificados os pontos de chamada no fluxo básico?	×		
8. Fluxos de exceção (descrição)	Todos os fluxos de exceção (se houver) estão numerados sequencialmente e descrevem claramente o diálogo entre o ator e o sistema em uma condição de exceção?	×		
9. Fluxos de exceção (chamadas)	Para todos os fluxos de exceção estão identificados os pontos de chamada, seja no fluxo básico, seja nos fluxos alternativos?	×		
10. Pós-condições	As pós-condições (se houver) estão descritas corretamente?			×

Faz parte da análise também a preocupação com a especificação dos requisitos não funcionais, que são chamados de requisitos de qualidade. No nosso sistema exemplo, um desses requisitos trata da facilidade de uso que o aplicativo deve ter, uma vez que, de acordo com o RNF-002 — O sistema deverá permitir que um usuário sem treinamento consiga operar qualquer função em menos de 3 minutos. Esse é um requisito considerado de usabilidade, ou seja, ele precisa compreender o funcionamento do aplicativo sem receber treinamento e realizar as funções sem ajuda. Sua especificação se encontra no Quadro 6. Nesse caso específico, foi estabelecido que, ao submeter o aplicativo para um conjunto de usuários típicos, 90% deles deve ser capaz de realizar as funções descritas.

Quadro 6. Especificação de um requisito não funcional

Identificador	Detalhamento
RNF-002	O sistema deverá permitir que um usuário sem treinamento consiga operar qualquer função em menos de 30 segundos. ■ Um usuário típico do sistema deverá ser capaz de compreender o funcionamento do sistema sem necessitar de treinamento. ■ Ao realizar o teste com 10 usuários típicos, 90% tem que ser capaz de consultar as multas sem a necessidade de acionar o botão de Ajuda. ■ Ao realizar o teste com 10 usuários típicos, 90% deve ser capaz de finalizar o pagamento de uma multa sem a necessidade de acionar o botão de Ajuda. (…)

Para realizar a verificação dos requisitos não funcionais, aplica-se o *checklist* referente aos requisitos, que é o mesmo apresentado anteriormente para avaliar os requisitos funcionais. Seu resultado de aplicação encontra-se no Quadro 7. Todos os itens do *checklist* foram atendidos, com exceção do item 6, pois a prioridade do requisito não está identificada. O item 2 não se aplica, pois quem pode analisá-lo é o usuário, e não um par do analista de requisitos, como é o caso da verificação.

Quadro 7. *Checklist* para a lista de requisitos – aplicado ao requisito RNF-002

RNF-002. O sistema deverá permitir que um usuário sem treinamento consiga operar qualquer função em menos de 3 minutos				
Atributo	**Definição**	**Sim**	**Não**	**Não se aplica**
1. Completo	O requisito está especificado de forma completa e que possibilita que o desenvolvedor o implemente?	×		
2. Correto	O requisito reflete o que o usuário, cliente ou seus representantes desejam?			×
3. Único	O requisito descreve uma única capacidade, característica, restrição ou atributo de qualidade?	×		
4. Viável	O requisito é viável técnica e financeiramente de ser implementado, de acordo com as restrições do projeto?	×		
5. Necessário	O requisito tem um motivo de existir, que é representado pelo seu relacionamento a uma fonte de informação e a um objetivo de negócio?	×		
6. Priorizado	O requisito tem uma prioridade atribuída para que possa ser alocado a uma versão do *software*?		×	
7. Não ambíguo	O requisito não contém ambiguidades que levem os *stakeholders* a interpretá-lo de forma diferente?	×		
8. Verificável	O requisito é possível de ser verificado posteriormente quanto à sua implementação?	×		
9. Conforme	O requisito está em conformidade com os padrões de especificação estabelecidos?	×		
Observações: nenhuma				

Fonte: Adaptado de Wiegers e Beatty (2013) e ISO/IEC/IEEE (2018).

Quando a equipe de desenvolvimento utiliza **métodos ágeis**, é comum que os requisitos funcionais sejam expressos como histórias de usuário. As histórias de usuário são compostas por cartões, com uma declaração da necessidade do usuário, e por critérios de aceitação, que normalmente detalham a história ou especificam os requisitos não funcionais (PATTON, 2014).

O responsável pelos itens que compõem o *product backlog*, e que irá gerar as histórias, é o *product owner*, ou simplesmente PO. Cabe a ele representar os interesses do cliente ou do usuário junto à equipe de desenvolvimento. Nesses casos também pode ser aplicado um *checklist* de histórias de usuário, que se assemelhará àquele apresentado no Quadro 2, referente aos requisitos funcionais.

Discutimos nesta seção a realização das atividades referentes à verificação de requisitos usando *checklists* a serem aplicados no formato de revisão por pares, ou seja, um colega, com capacitação igual ou superior ao autor do artefato, realiza a análise do artefato. Outra técnica que é usada para realizar a verificação de requisitos é o teste de *software*. Nesse caso, é necessário que o produto esteja implementado e que possa ser avaliado. O artefato que será usado como base para realizar os testes é o caso de teste. **Casos de teste** podem ser gerados assim que os requisitos estejam especificados. Eles podem ser documentos criados no editor de texto ou podem ser programas de teste automatizado. Se forem documentos, eles também podem ser revisados por pares, com o apoio de *checklists*.

Validação de requisitos

De acordo com a norma internacional ISO/IEC/IEEE 12207 (ISO/IEC/IEEE, 2017, p. 10–11, tradução nossa), a validação pode ser compreendida com sendo "[...] a confirmação, por meio do fornecimento de evidência objetiva, que o requisito foi atendido para um uso ou aplicação pretendidos específicos". De acordo com o modelo de maturidade CMMI-DEV 2.0 (CMMI INSTITUTE, 2018, p. 525, tradução nossa), a validação "[…] demonstra que a solução vai atender seu uso pretendido no ambiente alvo, isto é, 'você está construindo a coisa certa'".

A validação de requisitos precisa ser planejada prevendo o envolvimento dos *stakeholders* relevantes do projeto. Geralmente, ela é realizada por meio de análise dos artefatos apresentados, considerando a ótica de quem vai usar o produto no ambiente-alvo, o que é uma perspectiva um pouco diferente de quando inspecionamos o artefato sob a ótica técnica.

No caso do nosso exemplo, teriam que ser envolvidos pelo menos os seguintes *stakeholders*:

- coordenador da equipe de infrações;
- coordenador da equipe de emissão de carteiras de habilitação;
- coordenador da equipe de relacionamentos institucionais (lembre-se que o sistema terá interface com clínicas médicas e com os bancos).

Poderiam ainda ser envolvidos representantes das classes de usuários finais do produto, como o motorista e o proprietário. Também poderiam ser envolvidos os agentes de trânsito.

2 Matriz de rastreabilidade bidirecional

As atividades relacionadas ao gerenciamento de requisitos estão representadas na Figura 4. Elas se referem às preocupações que devemos ter ao longo de todo o ciclo de vida do produto, desde a sua concepção até a sua retirada de operação, ou descontinuação. Gerenciar os requisitos significa ter a visibilidade e o controle sobre o seu ciclo de vida.

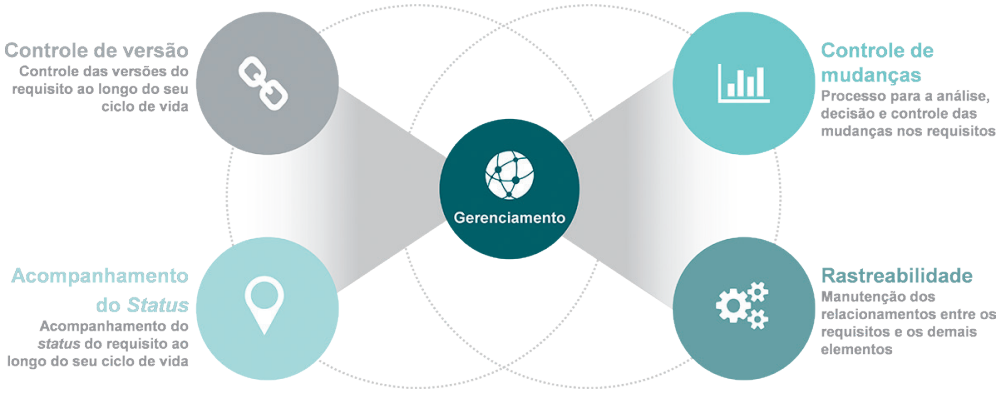


Figura 4. Atividades do gerenciamento de requisitos.

Nesta seção vamos focar na atividade de manutenção da **rastreabilidade** dos requisitos. A rastreabilidade é o mecanismo por meio do qual conseguimos ter elementos para analisar o impacto de uma solicitação de mudança.



Fique atento

Rastreabilidade é definida pela norma internacional ISO/IEC/IEEE 12207:2017(E), como o "[...] grau de relacionamento que pode ser estabelecido entre duas ou mais entidades lógicas, especialmente entidades que têm um relacionamento predecessor-sucessor ou superior-subordinado entre si, como requisitos, elementos do sistema, verificações ou tarefas." (ISO/IEC/IEEE, 2017, p. 10, tradução nossa).

O SWEBOK v3 define rastreabilidade da seguinte forma: "O rastreamento de requisitos diz respeito à recuperação da fonte dos requisitos e à previsão dos efeitos dos requisitos." (BOURQUE; FAIRLEY, 2014, documento *on-line,* tradução nossa).

O CMMI-DEV v1.3 define a rastreabilidade bidirecional como sendo "Uma associação entre duas ou mais entidades lógicas que é detectável em qualquer direção" (CMMI PRODUCT TEAM, 2010, documento *on-line,* tradução nossa).

A rastreabilidade se refere a rastrear um requisito à sua origem, o que é chamado de rastreabilidade para trás (*backward traceability*) e também aos seus desdobramentos, o que é chamado de rastreabilidade para frente (*forward traceability*). Essa é a chamada rastreabilidade vertical (BOURQUE; FAIRLEY, 2014). Existe também a rastreabilidade entre os requisitos (POHL; RUPP, 2015), que mapeia as dependências entre os requisitos, ou seja, de um requisito para outro requisito. Esta é chamada de rastreabilidade horizontal.

A Figura 5 ilustra um exemplo genérico de matriz, apenas para que seja possível compreender esses relacionamentos. Nela podemos identificar em destaque a rastreabilidade do requisito RF20 para todos os demais elementos do sistema.

Como você pode observar, a metade de baixo da matriz está pintada de cinza, pois essa parte não será preenchida, uma vez que é o espelho da parte de cima. Mapear os dois lados é redundante e pode levar a inconsistências quando se esquece de atualizar alguma das partes. Da mesma forma, caso haja intenção de não rastrear algum par, esse também deverá ser desabilitado. É o caso do canto superior direito, que também está pintado de cinza, indicando que não será analisada a rastreabilidade entre os casos de uso, casos de teste e módulos em relação a elementos organizacionais, como solicitante e objetivos estratégicos (OE).

	Requisitos funcionais			Requisitos não funcionais			Casos de Uso			Casos de Teste			Módulos		
	RF01	(...)	RF20	RNF1	RNF2	RNF3	UC1	(...)	UC12	CT1.1	(...)	CT20.1	M1	(...)	M54
Gerente de Vendas			X			X									
Diretor de Vendas	X														
Vendedor															
OE1	X														
OE2															
OE3			X												
RF01					X		X			X					X
(...)															
RF20						X			X			X	X		
RNF1															
RNF2															
RNF3															
UC1															
(...)															
UC12															
CT1.1															
(...)															
CT20.1															
M1															
(...)															
M54															

Figura 5. Exemplo de matriz de rastreabilidade de requisitos usando planilha eletrônica.

É comum que a matriz mapeie a rastreabilidade entre as fontes de informação (solicitante do requisito), os requisitos funcionais, os requisitos não funcionais, os casos de uso, os casos de teste e o código. A empresa pode decidir que elementos ela pretende manter relacionados. Pode ainda decidir que não vai rastrear todos os requisitos, mas que vai focar naqueles que são mais críticos para o negócio ou que representam mais danos para a arquitetura da aplicação.

A Figura 6 apresenta a matriz de rastreabilidade que foi construída para o exemplo do aplicativo do Detran Inteligente. Por uma questão de espaço, ela foi simplificada, mostrando apenas uma parte dos relacionamentos. Foi mapeada apenas a rastreabilidade vertical de requisitos funcionais em relação aos casos de uso. Foram considerados todos os 30 requisitos funcionais que foram apresentados no Quadro 1 e os 13 casos de uso que estão representados no diagrama da Figura 3.

Com o mapeamento da rastreabilidade já é possível perceber que os requisitos destacados em verde não têm caso de uso correspondente, ou seja, eles ainda não estão especificados, estão apenas listados na lista de requisitos funcionais. Todos os casos de uso, por sua vez, têm relacionamento mapeado com pelo menos um requisito funcional.

		UC1 Consultar multas	UC2 Consultar taxas	UC3 Agendar exame	UC4 Pagar taxas	UC5 Pagar multas	UC6 Consultar pontuação	UC7 Agendar biometria	UC8 Realizar login	UC9 Cadastrar veículo	UC10 Indicar condutor	UC11 Consultar IPVA	UC12 Pagar IPVA	UC13 Realizar cadastro
RF-001	O sistema deve possuir um cadastro de usuário													x
RF-002	Todos os usuários devem se logar para operar qualquer funcionalidade no aplicativo								x					
RF-003	O proprietário poderá cadastrar um ou mais veículos									x				
RF-004	O proprietário poderá indicar outro condutor, no caso de uma multa										x			
RF-005	O motorista poderá visualizar a sua CNH													
RF-006	O motorista poderá consultar seus pontos na CNH						x							
RF-007	O motorista poderá consultar suas multas pagas e pendentes	x												
RF-008	O motorista poderá recorrer de uma multa, apresentando justificativas													
RF-009	O motorista poderá dar início ao processo de 1ª habilitação													
RF-010	O motorista poderá realizar o pagamento das taxas para habilitação		x		x									
RF-011	O motorista poderá agendar exame nas clínicas de saúde conveniadas			x										
RF-012	A clínica de saúde poderá consultar a agenda de exames													
RF-013	A clínica poderá enviar aviso de cancelamento de consulta para o motorista													
RF-014	O motorista poderá enviar aviso de cancelamento de consulta para a clínica													
RF-015	O motorista poderá agendar biometria para renovação de CNH							x						
RF-016	O motorista poderá informar a sua chegada ao DETRAN													
RF-017	O proprietário poderá tirar extrato de multas dos veículos cadastrados em seu nome	x												
RF-018	O proprietário poderá emitir a guia do IPVA											x		
RF-019	O proprietário poderá pagar o IPVA											x	x	
RF-020	O proprietário poderá emitir a guia de licenciamento												x	
RF-021	O proprietário poderá pagar a guia de licenciamento													
RF-022	O DETRAN poderá disponibilizar agenda de biometria													
RF-023	O DETRAN poderá cancelar agenda de biometria até 24 horas antes da coleta													
RF-024	O agente de trânsito poderá aplicar uma multa ao veículo													
RF-025	O agente de trânsito poderá consultar dados do veículo pela placa													
RF-026	O proprietário poderá pagar taxas e multas usando cartão de débito				x	x								
RF-027	O proprietário poderá pagar taxas e multas usando cartão de crédito				x	x								
RF-028	O proprietário poderá pagar taxas e multas usando boleto				x	x								
RF-029	Os bancos conveniados devem passar diariamente um arquivo com os retornos de cobrança													
RF-030	O sistema deverá bloquear automaticamente a CNH de motoristas que excedam os pontos													

Figura 6. Matriz de rastreabilidade de requisitos do aplicativo Detran Inteligente.

A rastreabilidade dos requisitos vai garantir que (CMMI INSTITUTE, 2018):

- os requisitos foram totalmente tratados nos entregáveis para o cliente;
- os requisitos de baixo nível podem ser rastreados a uma fonte válida (solicitante);
- os requisitos foram implementados;
- os requisitos selecionados foram verificados e validados;
- os relacionamentos com outras entidades, como produtos de trabalho intermediários e finais, podem:
  - incluir dependências nas quais a mudança em um requisito pode implicar na mudança em outros requisitos;
  - apoiar-se no projeto (*design*) e na avaliação do impacto das mudanças;
  - afetar a avaliação do impacto das mudanças;
  - apoiar a mudança antecipada do escopo.
- o projeto (*design*) e outras documentações refletem os requisitos;
- os planos de teste e casos de teste refletem os requisitos.

3 Impacto de solicitações de mudança

Um dos usos mais relevantes da rastreabilidade é apoiar a análise das solicitações de mudanças. As mudanças certamente acontecerão, e é importante manter um processo que possa ser usado para analisar os impactos e tomar as decisões. Dependendo do porte do projeto ou do produto, essa análise é realizada diretamente pelo analista de requisitos. Mas, em casos de produtos maiores ou mais críticos, essa atribuição cabe ao Comitê de Controle de Mudanças (CCM). Esse comitê pode ser constituído de algumas poucas pessoas ou pode ser composto de um conjunto maior e mais representativo. A forma, o tamanho, a periodicidade das reuniões, os procedimentos para análise e decisão podem variar bastante.

Olhando o nosso exemplo do aplicativo Detran Inteligente, suponha que em uma reunião realizada com a equipe de agentes de trânsito foram identificados diversos novos detalhes que precisam fazer parte dos requisitos funcionais RF-24 e RF-25. A primeira providência é voltar na matriz representada na Figura 5 e analisar os relacionamentos que esses requisitos têm. Como você pode observar, eles não estão relacionados ainda a nenhum caso de uso. Supondo que a matriz está atualizada, isso quer dizer que esses requisitos não trarão nenhum impacto adicional além de serem revisados na lista de requisitos.

Se por acaso chegar uma solicitação de mudança para o RF-10, então, será necessário avaliar se haverá mudanças no UC2 e no UC4. Nesse caso, cabe à equipe analisar qual a situação atual desses requisitos (*status*). Dependendo da situação de um requisito, o retrabalho pode ser grande. Se o requisito estiver apenas especificado, o trabalho que se perde é apenas o do analista de requisitos. Se ele já estiver na etapa de homologação pelo cliente, esse retrabalho será muito maior, pois já terão sido gastas horas do analista que especificou, do seu colega que fez a revisão por pares, do analista de teste que especificou e realizou os testes e do implementador que codificou. Isso sem contar que outros papéis podem estar envolvidos como o *designer*, o documentador, o arquiteto e assim por diante.

A atividade de análise de impacto pode ser muito custosa se não houver nenhum tipo de rastreabilidade, pois ela passa a contar apenas com a experiência da equipe em se lembrar dos possíveis impactos ou, pior, ter que entrar em cada código para ver onde haverá mudanças. Qualquer uma dessas duas opções oferece risco, pois partes importantes podem ser deixadas de lado e não serem alteradas junto com a mudança.

Referências

BOURQUE, P.; FAIRLEY, R. *SWEBOK*: guide to the software engineering body of knowledge: version 3. USA: IEEE Computer Society, 2014. Disponível em: https://www.computer.org/education/bodies-of-knowledge/software-engineering. Acesso em: 25 jun. 2020.

CMMI INSTITUTE. *CMMI model V2.0*. Pittsburgh: CMMI Institute, 2018.

CMMI PRODUCT TEAM. *CMMI® for Development, Version 1.3*. Hascom AFB: Software Engineering Institute, 2010. Disponível em: https://resources.sei.cmu.edu/asset_files/TechnicalReport/2010_005_001_15287.pdf. Acesso em: 22 jun. 2020.

ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017(E)*: systems and software engineering: software life cycle processes. Switzerland: ISO, 2017.

ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018:* systems and software engineering: life cycle processes: requirements engineering. Switzerland: ISO, 2018.

PATTON, J. *User story mapping*: discover the whole story, build the right product. Sebastopol: O´Reilly, 2014.

POHL, K.; RUPP, C. *Requirements engineering fundamentals:* a study guide for the certified professional for requirements engineering exam, foundation level, IREB Compliant. 2. ed. Santa Barbara: Rock Nook, 2015.

PREZADO ESTUDANTE

ENCERRA AQUI O TRECHO DO LIVRO DISPONIBILIZADO PELA SAGAH PARA ESTA PARTE DA UNIDADE.

CONTRIBUA COM A QUALIDADE DO SEU CURSO

Se você encontrar algum problema neste material, entre em contato pelo email **eadproducao@unilasalle.edu.br.** Descreva o que você encontrou e indique a página.

Lembre-se: a boa educação se faz com a contribuição de todos!

UNIVERSIDADE
LaSalle
Viver é evoluir.

Av. Victor Barreto, 2288 | Canoas - RS
CEP: 92010-000 | 0800 541 8500
eadproducao@unilasalle.edu.br